



Inferring Evolutionary Parameters from Phylogenies

Honours Thesis

School of Computer Science and Engineering
University of New South Wales

Submitted: November 2025

Student ID: z5405719

Supervisor: Dr Mark Tanaka

Assessor: Prof Sara Ballouz

Acknowledgements

I would like to thank my supervisor, **Dr Mark Tanaka**, for his supervision, guidance, and patient explanations throughout this project. I am also grateful to **Prof Sara Ballouz** for her valuable suggestions and feedback.

Abstract

Inferring the parameters of simple evolutionary models from phylogenetic trees is fundamental but challenging, largely because the likelihood of a tree under a given demographic history is analytically intractable. Modern simulation-based methods based on Neural Posterior estimation offer a principled way to approximate posterior distributions, but their performance depends critically on how trees are represented as inputs to the model. In this thesis I study a four-parameter coalescent bottleneck model (N_e, g, t_b, f_b) and compare three ways of turning phylogenies into fixed-length vectors. Using large number of simulated trees, I train Neural Posterior Estimation based on Normalizing Flows on each representation and evaluate them on several held-out parameter settings, measuring absolute and relative errors in both parameter space and summary-statistic space, as well as inspecting full posterior marginals.

Contents

List of Figures	5
List of Tables	7
1 Introduction	8
1.1 Research Background	8
1.2 Problem Statement	8
1.3 Hypotheses	8
2 Literature Review	9
2.1 Foundations: From Evolutionary Models to Phylogenetic Trees	9
2.2 The Intractable Likelihood Problem	11
2.3 First-Generation Solutions: Approximate Bayesian Computation (ABC) .	11
2.4 Deep Learning for Point Estimates	13
2.5 The Modern Frontier: Neural Posterior Estimation (NPE)	15
2.6 Graph Neural Networks for Graph-Structured and Biological Data	18
2.7 Summary and Motivation	19
3 Methodology	19
3.1 Demographic model and choice of parameters	20
3.2 Representing phylogenies for inference	24
3.3 Neural Posterior Estimation (NPE)	33
3.4 Training vs. testing	35
4 PhyloInference Dashboard	36
5 Evaluation	37
5.1 Parameter-space errors	37
5.2 Summary-statistic errors	38
5.3 Qualitative inspection of posterior distributions	39
6 Results	40
6.1 Parameter setting 1: Moderate size, early bottleneck	40
6.2 Parameter setting 2: Recent severe bottleneck with growth	44
6.3 Parameter setting 3: Older moderate bottleneck with growth	50
6.4 Parameter setting 4: Large population with very recent severe bottleneck	54
7 Conclusion	59
8 Future work	60

References	61
Appendix A: Code Listings	64

List of Figures

List of Figures

1	Wright–Fisher process schematic	10
2	Binary-trait phylogenetic tree construction	10
3	Approximate Bayesian Computation (ABC) flowchart. A parameter vector $\theta \sim p(\theta)$ is used in a simulator to generate x' , which is compared to the observed data x . If $\rho(x', x) \leq \varepsilon$, the θ is accepted as a (weighted) posterior sample. Taken from Cranmer et al. (2020).	12
4	Deep FFNN for summary-statistic inference	14
5	Convolutional neural network approach for population genetic inference. A genotype matrix is treated as an image and passed through convolutional and pooling layers that automatically learn informative features. Taken from Flagel et al. (2019).	15
6	Case-based vs. globally amortized inference. Case-based approaches (e.g. ABC) repeat the full simulation procedure for every new dataset, whereas amortized methods (e.g. BayesFlow) pay the simulation cost upfront and then enable fast inference for new data. Taken from Radev et al. (2020).	16
7	Comparison of architectures. Left: discrete transformations in standard normalizing flows (NPE). Right: continuous trajectories in flow matching (FMPE). Taken from Dax et al. (2023).	17
8	Statistics of published graph neural network (GNN) articles in bioinformatics from 2015 to 2020. The orange bars show the total number of GNN papers in bioinformatics for each year (with 2020 counted up to October only), while the remaining colours indicate the contributions from disease association prediction, drug research, and medical image processing, respectively, which together form the total, Figure 2 in (Zhang et al. 2021).	18
9	Overall SBI pipeline for phylogenetic inference	20
10	Effect of N_e on coalescent tree height	22
11	Effect of growth rate g on tree shape	23
12	Bottleneck timing and severity	24
13	Example tree for handcrafted statistics	26
14	GIN encoder for tree embeddings	30
15	GIN layer update for a single node	30
16	Deep Graph Infomax training scheme	32
17	Normalising-flow view of NPE	33
18	PhyloInference Play dashboard	37

19	Phylogeny simulated at $\theta_{\text{true}} = (5000, 0, 1000, 0.1)$	40
20	Lineage-through-time plot for the tree in Figure 19.	41
21	Posterior marginal distributions for (N_e, g, t_b, f_b) using the summary-statistics embedding.	41
22	Posterior marginals using the unsupervised GIN embedding.	42
23	Posterior marginals using the supervised GIN embedding.	43
24	True phylogeny corresponding to Parameter setting 2.	45
25	Lineage-through-time (LTT) plot for the same tree as in Figure 24. . . .	45
26	Posterior marginal distributions for (N_e, g, t_b, f_b) using the summary statistics embedding (Parameter setting 2).	46
27	Posterior marginal distributions for (N_e, g, t_b, f_b) using the unsupervised GIN embedding (Parameter setting 2).	47
28	Posterior marginal distributions for (N_e, g, t_b, f_b) using the supervised GIN embedding (Parameter setting 2).	48
29	Phylogeny simulated at $\theta_{\text{true}} = (20,000, 0.015, 800, 0.05)$	50
30	Lineage-through-time plot for the tree in Figure 29.	51
31	Posterior marginal distributions for (N_e, g, t_b, f_b) using the summary-statistics embedding.	51
32	Posterior marginals using the unsupervised GIN embedding.	52
33	Posterior marginals using the supervised GIN embedding.	53
34	Phylogeny simulated at $\theta_{\text{true}} = (25,000, 0.02, 200, 0.01)$	55
35	Lineage-through-time plot for the tree in Figure 34.	55
36	Posterior marginal distributions for (N_e, g, t_b, f_b) using the summary-statistics embedding.	56
37	Posterior marginals using the unsupervised GIN embedding.	57
38	Posterior marginals using the supervised GIN embedding.	58

List of Tables

List of Tables

1	Comparison of mean error for population size inference, taken from Sheehan and Song (2016).	14
2	High-level comparison of approaches.	16
3	Summary statistics for the tree in Figure 19.	43
4	Absolute parameter errors for Parameter setting 1.	43
5	Absolute errors in summary statistics for Parameter setting 1.	43
6	Relative parameter errors (percent) for Parameter setting 1.	44
7	Relative errors (percent) in summary statistics for Parameter setting 1. .	44
8	True summary statistics for the tree in Parameter setting 2.	48
9	Absolute parameter errors for Parameter setting 2. For each model we report $ \Delta N_e $, $ \Delta g $, $ \Delta t_b $, $ \Delta f_b $, and the total absolute parameter error $\sum_j \Delta \theta_j $	48
10	Absolute errors in summary statistics for Parameter setting 2. We report $ s_{\text{pred}} - s_{\text{true}} $ for each statistic and the total absolute error $\sum_k s_k $	49
11	Relative parameter errors (absolute error divided by the true value) for Parameter setting 2, reported as percentages.	49
12	Relative errors in summary statistics for Parameter setting 2, reported as percentages $ s_{\text{pred}} - s_{\text{true}} / s_{\text{true}} $	49
13	Summary statistics for the tree in Figure 29.	53
14	Absolute parameter errors for Parameter setting 3.	53
15	Absolute errors in summary statistics for Parameter setting 3.	53
16	Relative parameter errors (percent) for Parameter setting 3.	54
17	Relative errors (percent) in summary statistics for Parameter setting 3. .	54
18	Summary statistics for the tree in Figure 34.	58
19	Absolute parameter errors for Parameter setting 4.	58
20	Absolute errors in summary statistics for Parameter setting 4.	58
21	Relative parameter errors (percent) for Parameter setting 4.	59
22	Relative errors (percent) in summary statistics for Parameter setting 4. .	59

1 Introduction

1.1 Research Background

The effort to reconstruct the past from present-day genetic data is a central theme in evolutionary biology. The parameters of our evolutionary models population sizes, migration rates, selection coefficients—are the quantitative signatures of the historical processes that generated the diversity we see today. For years, the primary obstacle to estimating these parameters has been the intractable likelihood problem. This challenge gave rise to a powerful set of techniques known as simulation-based inference (SBI). The foundational method in this family is Approximate Bayesian Computation (ABC), which sidesteps the likelihood calculation entirely by running many simulations with random parameters and retaining those that closely match the observed data Beaumont et al. (2002). While revolutionary, simple rejection ABC is inefficient. This led to crucial improvements, notably Sequential Monte Carlo (SMC) for ABC, which evolves a population of parameter “particles” over rounds to converge more efficiently (Sisson et al. 2007).

More recently, deep learning offered a different path: neural networks trained on large libraries of simulated genetic data to learn the inverse mapping from data to parameters. These methods are fast and powerful but typically provide point estimates rather than full posteriors (Sheehan and Song 2016). This motivates unifying the statistical rigor of ABC with the speed of deep models. In this regime, Neural Posterior estimation has become popular (Cranmer et al. 2020).

1.2 Problem Statement

Theoretical advantages of Neural Posterior Estimation (NPE) with flexible density models such as normalizing flows are well understood, but their behaviour in phylogenetic applications is not. In particular, it is unclear how different ways of representing trees as fixed-length vectors affect which demographic parameters can be reliably recovered. At the same time, general-purpose deep learning methods on trees often prioritise point prediction over calibrated uncertainty. There is therefore a need to see how alternative tree representations (hand-crafted summary statistics versus graph neural network embeddings) interact with NPE to shape posterior accuracy and calibration.

1.3 Hypotheses

This research is designed to test the practical utility of applying modern NPE methods to phylogenetic trees under a bottleneck model, with a focus on representation learning. Concretely, we hypothesise that:

- (H1) The choice of tree representation has a substantial impact on which aspects of the demography (N_e, g, t_b, f_b) are identifiable and how sharply they can be inferred.
- (H2) Unsupervised GIN+DGI and Supervised GIN embeddings of trees will better preserve information about the overall shape and balance of genealogies than a small hand-crafted summary-statistic vector, leading to improved posterior recovery for some parameters.

1.3.1 Methodological Justification: Tree-Based Inference

We perform inference directly on phylogenetic trees rather than on raw genetic sequence alignments. While constructing a tree from sequence data has some information loss but also the rooted tree is the natural object generated by the coalescent and summarises the core genealogical history of the sampled individuals. Treating the tree as the primary data allows us to decouple tree reconstruction from demographic inference and to focus on the second stage: mapping trees to some form of embedding that could be a input to modern inference models.

2 Literature Review

2.1 Foundations: From Evolutionary Models to Phylogenetic Trees

At the core of population genetics are models describing how genetic composition changes over time. The Wright–Fisher model is a foundational forward-in-time model of random genetic drift: each new generation is formed by sampling parents with replacement from the previous generation (Fisher 1930; Wright 1931). Random sampling causes stochastic allele-frequency fluctuations (genetic drift), a primary driver of evolutionary change.

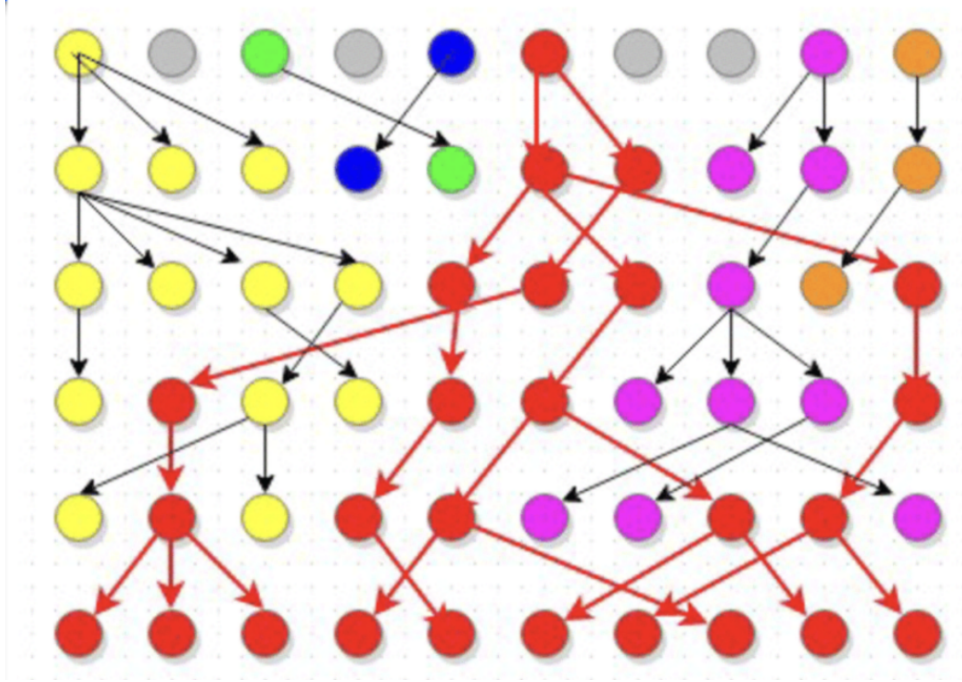


Figure 1: Wright-Fisher process schematic. Each row of dots represents a generation. Arrows trace parent-offspring relationships through time, showing how some lineages expand (e.g., red) while others go extinct (e.g., grey and green). *Source:* taken from Queau et al. (2016), Fig. 1.

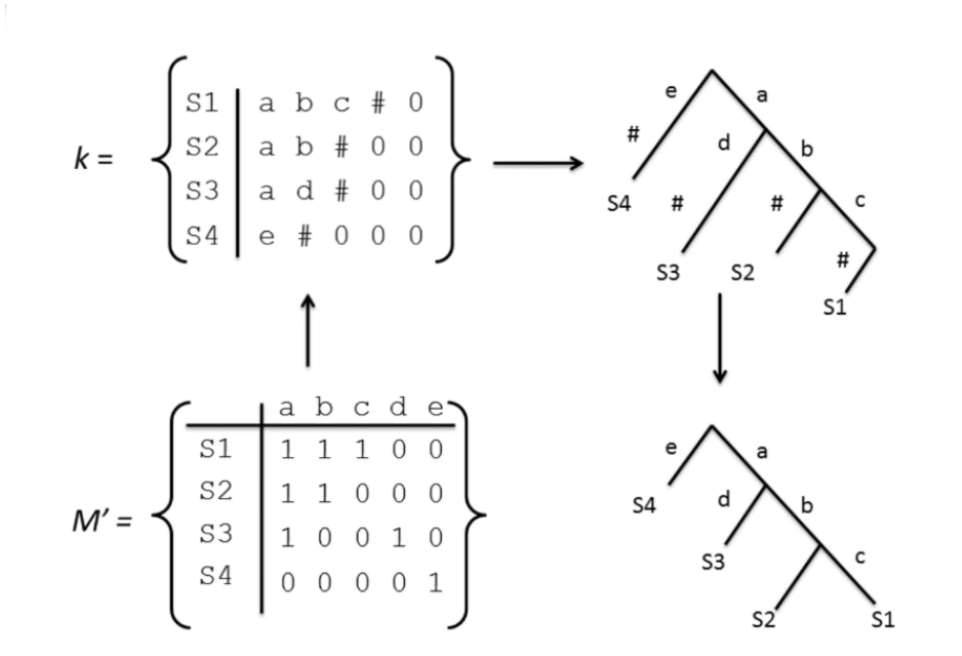


Figure 2: Binary-trait phylogenetic tree construction. *Source:* taken from Cmero (2015), *Building Phylogenetic Trees with Binary Traits*.

The historical record of this evolutionary process, combined with mutations that occur over time, is ultimately captured in the DNA of the individuals in the final generation.

This genetic data can be represented as a character matrix, where rows are individuals and columns are sites in the genome. From this matrix, we can then infer a phylogenetic tree, as shown in Figure 2.

Each tree represents a genealogy, tracing the lines of descent of a sample of individuals back through time to their common ancestors. The shape of a tree its topology and the lengths of its branches (representing time or genetic divergence) contains rich information about past demographic events like population bottlenecks or expansions. However, simulating the entire Wright-Fisher process forward in time is computationally very expensive, as it requires tracking every single individual in the population. For the purpose of inference, where we need to run thousands or millions of simulations, this is impractical. This is why we use a much more efficient backward-in-time framework called coalescent theory. The coalescent is a powerful mathematical approximation of the Wright-Fisher process that focuses only on the ancestry of the specific individuals we have sampled, ignoring all the irrelevant lineages that went extinct. Our project, therefore, uses the correct and standard approach: we use a coalescent simulator (msprime) because it is the most efficient way to generate the phylogenetic trees that would be expected under a Wright-Fisher-like evolutionary model.

2.2 The Intractable Likelihood Problem

While evolutionary models allow us to simulate evolution forwards, the central challenge of inference is working backward: given a phylogenetic tree, what were the parameters that generated it? The fundamental problem is that the likelihood function the probability of our data given a set of parameters is typically intractable as explained by Cranmer et al. (2020).

2.3 First-Generation Solutions: Approximate Bayesian Computation (ABC)

The first major breakthrough on the intractable-likelihood problem was Approximate Bayesian Computation (ABC). The core idea is simple: instead of evaluating the likelihood, draw parameters from the prior, simulate data, and retain those parameters whose simulations are close enough to the observation—typically judged via a distance on summary statistics (Pritchard et al. 1999; Beaumont et al. 2002). The basic workflow is illustrated in Figure 3.

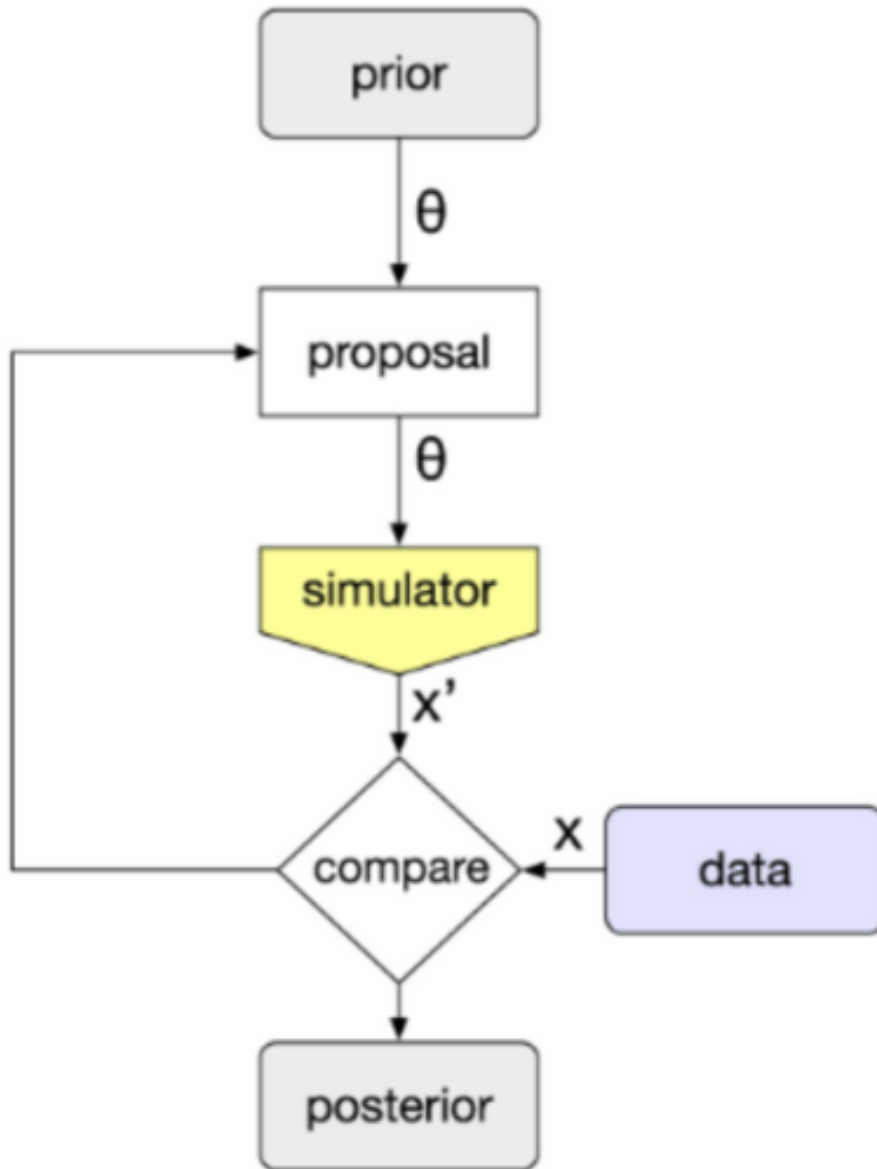


Figure 3: Approximate Bayesian Computation (ABC) flowchart. A parameter vector $\theta \sim p(\theta)$ is used in a simulator to generate x' , which is compared to the observed data x . If $\rho(x', x) \leq \varepsilon$, the θ is accepted as a (weighted) posterior sample. Taken from Cranmer et al. (2020).

The most basic version, *rejection ABC*, is brute-force and highly inefficient. This motivated smarter proposals. ABC with Markov chain Monte Carlo (ABC-MCMC) proposes new parameters near previously accepted ones, while *Sequential Monte Carlo* (ABC-SMC) evolves a population of parameter “particles” through increasingly strict thresholds, greatly improving efficiency over naive rejection (Sisson et al. 2007).

Despite these advances, classical simulation-based approaches face persistent limita-

tions (Cranmer et al. 2020):

- **Poor sample efficiency:** Obtaining accurate posteriors often requires prohibitively many simulations.
- **Information loss:** Compressing complex data into a few summary statistics can lose important structure of the data being inferred.
- **No amortization:** The costly simulation-and-fit loop must be repeated from scratch for every new observation, hindering scalability across many datasets.

2.4 Deep Learning for Point Estimates

The deep learning revolution offered powerful new methods for simulation-based inference. Machine learning, and specifically supervised learning, provides a framework for training algorithms to learn a mapping from complex inputs to desired outputs by showing them a vast number of labeled examples.

2.4.1 Learning from Summary Statistics: Feed-Forward Neural Networks (FFNNs)

Sheehan and Song (2016) were among the first to apply deep learning to population genetic inference, positioning it as a powerful alternative to ABC. The architecture of their model is shown in Figure 4.

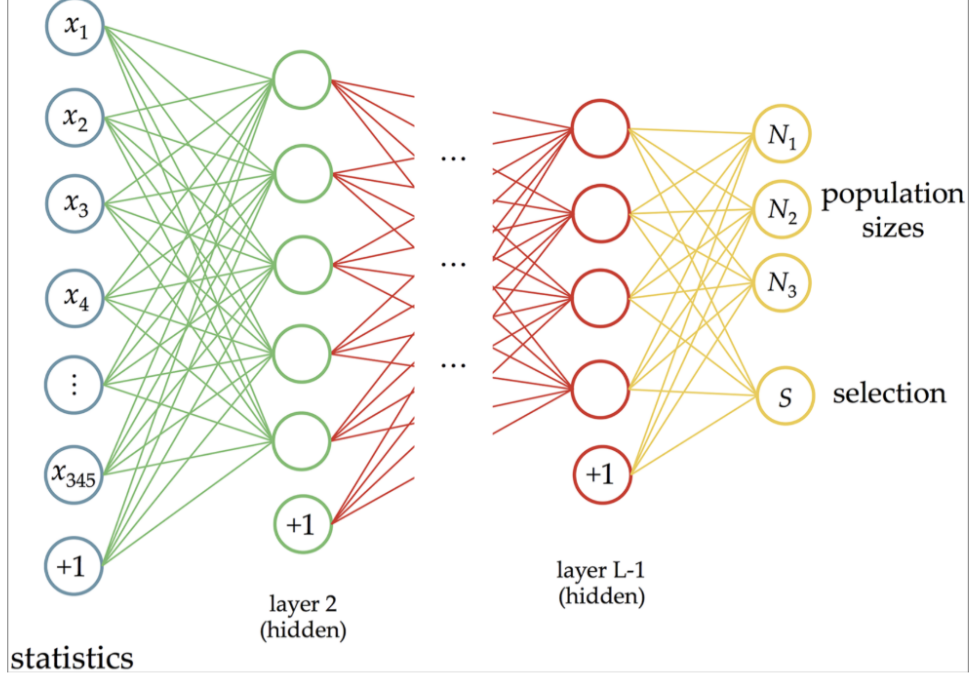


Figure 4: Deep feed-forward neural network for summary-statistic inference. The network takes a high-dimensional vector of summary statistics as input, processes them through multiple hidden layers, and outputs estimates for demographic parameters (N_1, N_2, N_3) and a selection class (S). *Source:* taken from Sheehan and Song (2016).

The methodology involved training the FFNN to learn the mapping from a high-dimensional vector of summary statistics to the evolutionary parameters of interest. Their model took 345 statistics as input including the site frequency spectrum (SFS), Tajima’s D, and measures of linkage disequilibrium (LD) which were processed through multiple hidden layers. The output layer jointly estimated a three-epoch demographic history (N_1, N_2, N_3) and classified the genomic region into a selection category (S).

The results, summarized in Table 1, showed that the deep learning approach was often more accurate than ABC, particularly when using the full set of summary statistics.

Table 1: Comparison of mean error for population size inference, taken from Sheehan and Song (2016).

Dataset / Method	N_1 error	N_2 error	N_3 error
Full stats / ABCtoolbox	0.062	0.043	0.218
Full stats / Deep learning	0.044	0.028	0.221
Filtered stats / ABCtoolbox	0.161	0.035	0.311
Filtered stats / Deep learning	0.065	0.055	0.319

Despite these advances, the method had important limitations. It still relied on hand-crafted summary statistics, which compress the raw sequence alignment and discard

spatial information that might otherwise improve inference accuracy.

2.4.2 Learning from Raw Data: Convolutional Neural Networks (CNNs)

Another approach set out to avoid hand-crafted summary statistics by learning directly from the raw multiple sequence alignment: the genotype matrix is viewed as a two-dimensional image and processed with convolutional neural networks (CNNs), which learn features end-to-end from the data (Flagel et al. 2019).

In this paradigm, a genotype matrix (with individuals as rows and genomic sites as columns) is sorted and fed into the CNN as a two-dimensional image (Figure 5). Instead of computing summary statistics beforehand, the CNN applies a series of learnable filters (kernels) that slide across this input image. These filters are trained to automatically detect salient spatial patterns directly from the data. For example, a filter might learn to recognize the characteristic long haplotype blocks of reduced diversity that are signatures of selective sweeps. By stacking multiple convolutional and pooling layers, the network builds a hierarchical representation of features, from simple local patterns to more abstract ones.

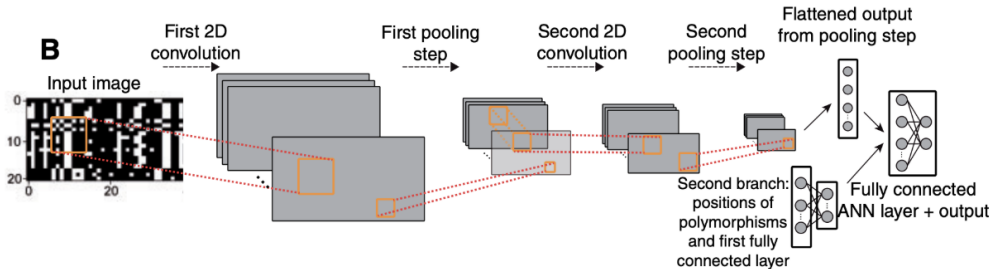


Figure 5: Convolutional neural network approach for population genetic inference. A genotype matrix is treated as an image and passed through convolutional and pooling layers that automatically learn informative features. Taken from Flagel et al. (2019).

This approach effectively automated the feature-extraction step that had been performed manually in earlier methods. Flagel et al. (2019) demonstrated that CNNs could solve a wide variety of population genetic inference tasks with high accuracy. However, despite their power, these methods shared the same limitation as earlier deep learning approaches: they provided only a single point estimate or classification, without the full posterior distributions needed for robust uncertainty quantification.

2.5 The Modern Frontier: Neural Posterior Estimation (NPE)

The limitations of both traditional statistical methods and early deep learning approaches created a clear need for a new paradigm. Neural Posterior Estimation (NPE) emerges as a modern synthesis, designed to bridge this divide. It provides the full posterior

distributions of Bayesian methods together with the speed and amortization benefits of deep learning (Cranmer et al. 2020).

A central concept of NPE is *amortized inference*. Here, the computationally expensive work running simulations and training a neural network is performed once, upfront. After training, the network serves as a reusable inference engine: when presented with new observations, it can instantly generate the corresponding posterior distribution. This contrasts sharply with ABC, where the costly simulation-and-comparison process must be repeated from scratch for each new dataset.

Table 2: High-level comparison of approaches.

Feature	ABC	Point-estimate DL	NPE
Primary output	Approx. posterior distribution	Single point estimate	Approx. posterior distribution
Uncertainty	Yes (spread of samples)	No	Yes (learned distribution)
Amortization	No; repeat for each dataset	Yes; inference is instant	Yes; inference is instant
Simulation cost	Paid at inference time	Paid upfront during training	Paid upfront during training

The fundamental difference in workflow between older case-based methods and the modern amortized approach is shown in Figure 6.

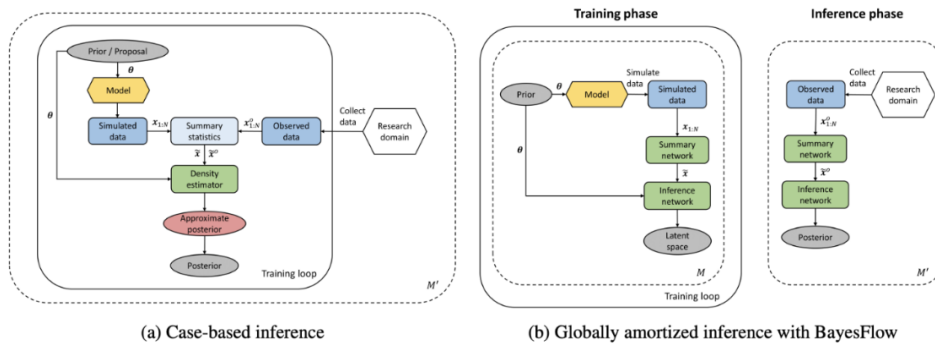


Figure 6: Case-based vs. globally amortized inference. Case-based approaches (e.g. ABC) repeat the full simulation procedure for every new dataset, whereas amortized methods (e.g. BayesFlow) pay the simulation cost upfront and then enable fast inference for new data. Taken from Radev et al. (2020).

Architectures for Neural Posterior Estimation. Within NPE, two prominent architectures are widely used:

Normalizing Flows (NPE). As described by Cranmer et al. (2020), normalizing flows are flexible neural networks that learn a bijective transformation from a simple base distribution (e.g. Gaussian) to the target posterior. This allows highly complex posterior shapes to be represented.

Flow Matching (FMPE). A more recent alternative is Flow Matching Posterior Estimation (FMPE). Instead of discrete bijections, FMPE trains a neural network to represent a continuous vector field transporting samples from the base distribution to the posterior (Dax et al. 2023). This often yields more efficient learning of complex, high-dimensional posteriors. However, inference requires solving an ordinary differential equation, which can be slower compared to a single forward pass through a normalizing flow.

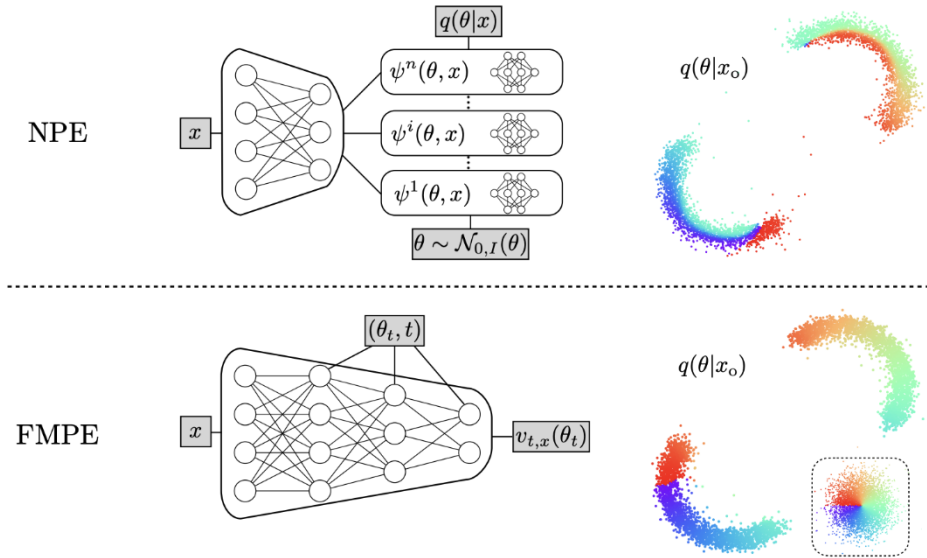


Figure 7: Comparison of architectures. Left: discrete transformations in standard normalizing flows (NPE). Right: continuous trajectories in flow matching (FMPE). Taken from Dax et al. (2023).

As Figure 7 illustrates, normalizing flows often require specialized, invertible architectures, while FMPE can use flexible networks such as simple multilayer perceptrons. This flexibility allows FMPE to better handle the high-dimensional and complex data encountered in phylogenetic inference, making it a promising candidate for scaling simulation-based inference to realistic evolutionary models.

2.6 Graph Neural Networks for Graph-Structured and Biological Data

Graph neural networks (GNNs) are a class of deep learning models designed for data that are naturally represented as graphs rather than in Euclidean space. They have emerged as a powerful tool for processing complex relational structures and have been widely adopted across bioinformatics as biological data increasingly take the form of networks (e.g. protein–protein interaction networks, gene regulatory networks, drug–target graphs) (Zhang et al. 2021). In essence, GNNs are connectionist models that capture dependencies in graph-structured data through iterative message passing between nodes, allowing them to aggregate information from local neighbourhoods while respecting the global topology.

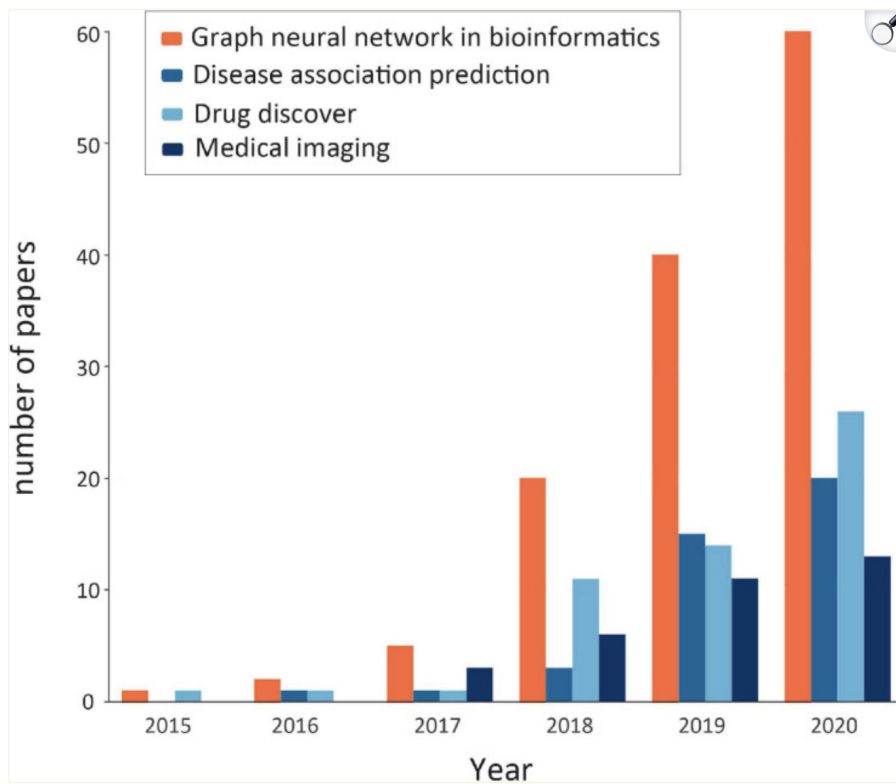


Figure 8: Statistics of published graph neural network (GNN) articles in bioinformatics from 2015 to 2020. The orange bars show the total number of GNN papers in bioinformatics for each year (with 2020 counted up to October only), while the remaining colours indicate the contributions from disease association prediction, drug research, and medical image processing, respectively, which together form the total, Figure 2 in (Zhang et al. 2021).

In practice, GNNs have demonstrated strong performance in a range of tasks examples include predicting molecular properties from chemical graphs (Duvenaud et al. 2015), extracting cross-sentence relations in text via graph-structured encoders (Peng et al. 2017), performing zero-shot recognition by combining semantic embeddings with knowledge graphs (X. Wang et al. 2018), and modelling drug interaction side effects using graph

convolutional networks on polypharmacy graphs (Zitnik et al. 2018).

2.7 Summary and Motivation

Coalescent theory provides a fast and principled way to simulate genealogies under Wright–Fisher-like models, making large-scale generation of phylogenetic trees computationally feasible. At the same time, Neural Posterior Estimation (NPE) has emerged as a modern simulation-based inference framework that can learn amortized posterior distributions for models with intractable likelihoods. In parallel, graph neural networks (GNNs) offer a natural way to embed graph-structured data, suggesting that phylogenetic trees can be converted into informative vector representations for inference. However, there has been relatively little work combining NPE with different tree representations, especially comparing handcrafted summary statistics to GNN-based embeddings. This thesis therefore focuses on using NPE together with both summary-statistic and GNN-based representations of phylogenetic trees to infer demographic parameters, and on evaluating how these choices affect the quality of the resulting posterior estimates.

3 Methodology

The methodology for this research is built upon the modern framework of simulation-based inference (SBI) using neural networks. The complete workflow, from drawing demographic parameters to obtaining a posterior distribution, is summarised in Figure 9. In broad terms, parameters (N_e, g, t_b, f_b) are sampled from their priors, used to simulate coalescent trees with `msprime`, transformed into vector representations, and finally passed to a neural posterior estimator that approximates $p(\theta | x)$.

Figure 9 therefore serves as the road map for this section. In the subsections that follow, we move from left to right through the pipeline: prior specification and parameter sampling, coalescent simulation with `msprime`, construction of summary-statistic and GNN-based representations of the trees, and finally training and evaluation of the neural posterior estimators.

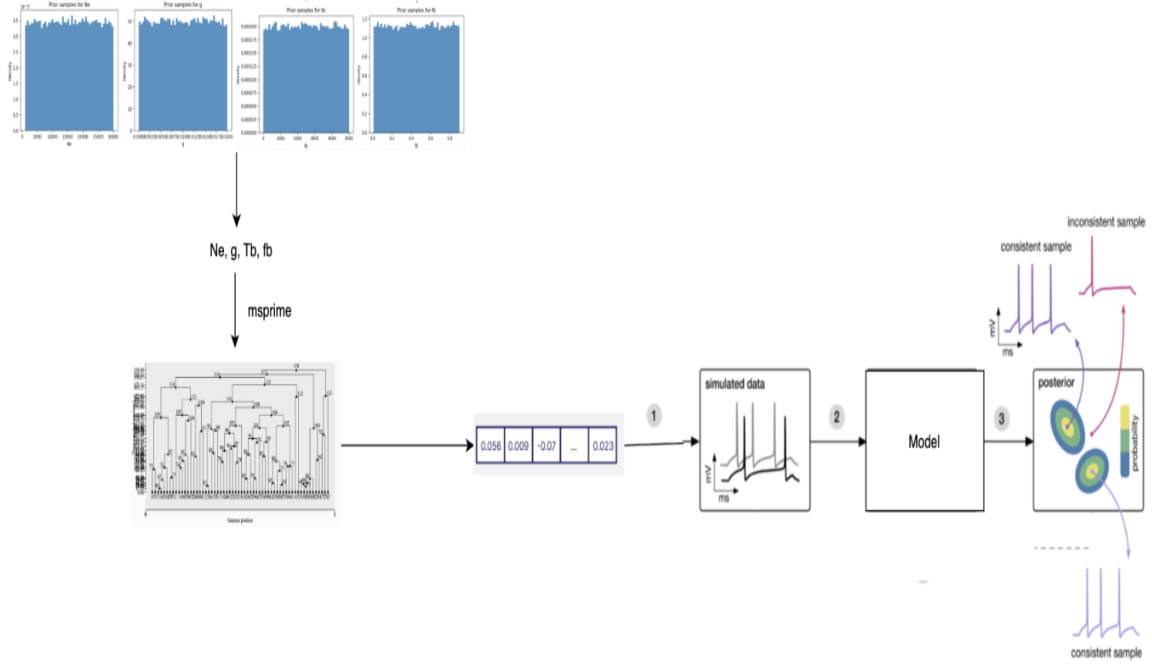


Figure 9: Overview of the simulation-based inference (SBI) pipeline used in this thesis. Demographic parameters (N_e, g, t_b, f_b) are sampled from their priors and passed to `msprime` to simulate a coalescent tree. The resulting tree is mapped to a fixed-dimensional representation (either handcrafted summary statistics or GNN-based embeddings), which is then used as input to a neural posterior estimator that approximates the posterior over demographic parameters.

Source: adapted from the general NPE workflow in the `sbi` (SBI Developers 2025) documentation.

3.1 Demographic model and choice of parameters

To generate training data, I simulate genealogical trees under the Kingman coalescent using a simple but flexible demographic model implemented in `msprime`. Rather than attempting to capture an arbitrarily complex population history, I focus on a small set of interpretable demographic parameters that strongly shape the structure of a phylogenetic tree.

I consider two related models:

- a **2-parameter model** with constant effective population size and exponential growth; and
- an **extended 4-parameter bottleneck model** that adds a transient reduction in population size.

Both models are parameterised by a vector of four demographic parameters

$$\theta = (N_e, g, t_b, f_b),$$

with independent uniform priors over a hyper-rectangular parameter space:

$$\begin{aligned} N_e &\sim \mathcal{U}(1,000, 30,000), \\ g &\sim \mathcal{U}(0, 0.02), \\ t_b &\sim \mathcal{U}(0, 5,000), \\ f_b &\sim \mathcal{U}(0.01, 0.9). \end{aligned}$$

These parameters are chosen for two main reasons:

1. They directly control key aspects of tree shape, so varying them across the prior produces qualitatively different genealogies.
2. They form a biologically meaningful yet low-dimensional parameterisation. Many real populations (for example, viral, bacterial, or human populations under expansion) are well approximated by long-term growth or decline together with one or more strong reductions in size. The model used here should be viewed as a simplified version of such histories but by adding further structure (for example, multiple bottlenecks or changes in growth rate at each bottleneck) we could obtain demographic scenarios that more closely resemble fully realistic population dynamics, while retaining the same basic modelling framework.

3.1.1 Effective population size N_e

In my simulations, N_e is the present-day effective population size of the single population modelled in `msprime`. Under the Kingman coalescent, time is scaled by $2N_e$ generations: with k ancestral lineages, the waiting time to the next coalescent event is exponential with rate

$$\lambda_k = \frac{\binom{k}{2}}{2N_e}.$$

Intuitively, a larger N_e makes coalescent events rarer, so lineages take longer to merge as we go backwards in time. This stretches the tree vertically and increases both the total tree height, time to the Most Recent Common Ancestor (MRCS) and the total branch length. Conversely, a smaller N_e increases the coalescent rate, compressing events closer to the present and producing shallower trees.

Figure 10 illustrates this effect by comparing two trees simulated with identical settings except for N_e : a larger N_e produces a much taller genealogy, while a smaller N_e yields a more recent MRCA and shorter overall branch lengths.

In the 2-parameter model (N_e, g) , N_e sets this global time scale while g controls how population size changes through time. In `msprime` I implement this by specifying a single population with `initial_size` = N_e and `growth_rate` = g ; for fixed g , changing N_e mainly rescales all branch lengths without strongly altering the tree topology. In the

4-parameter bottleneck model (N_e, g, t_b, f_b) , N_e still plays the role of the baseline size before and after the bottleneck, so it controls the overall height of the tree.

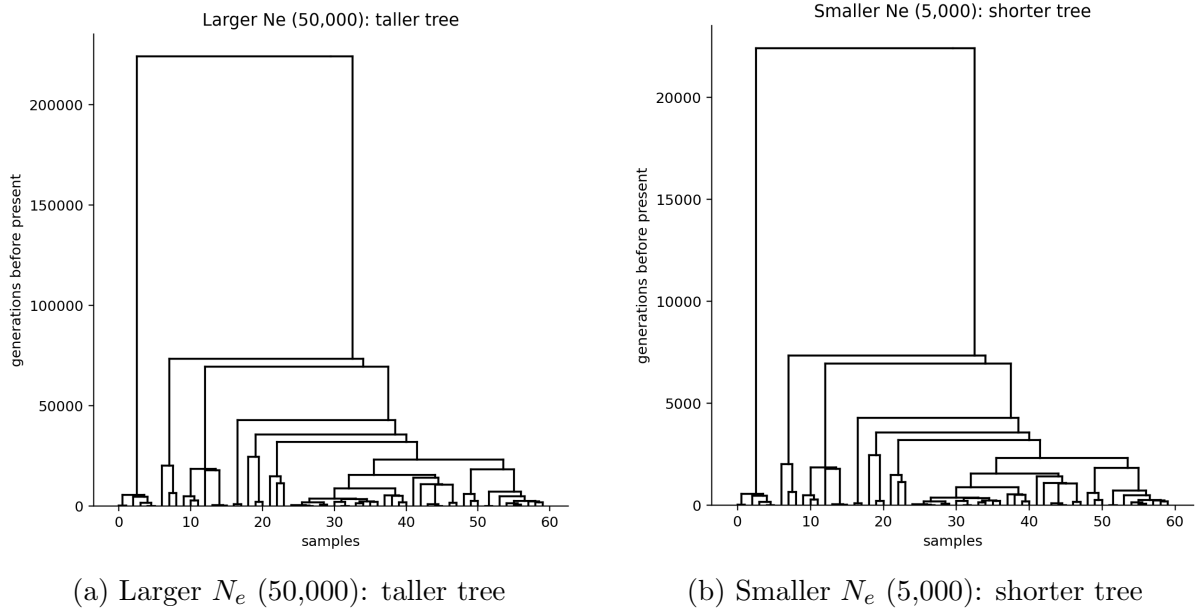


Figure 10: Effect of effective population size N_e on genealogical tree shape. Larger N_e leads to deeper, “taller” trees with older most recent common ancestors, whereas smaller N_e compresses coalescent events closer to the present.

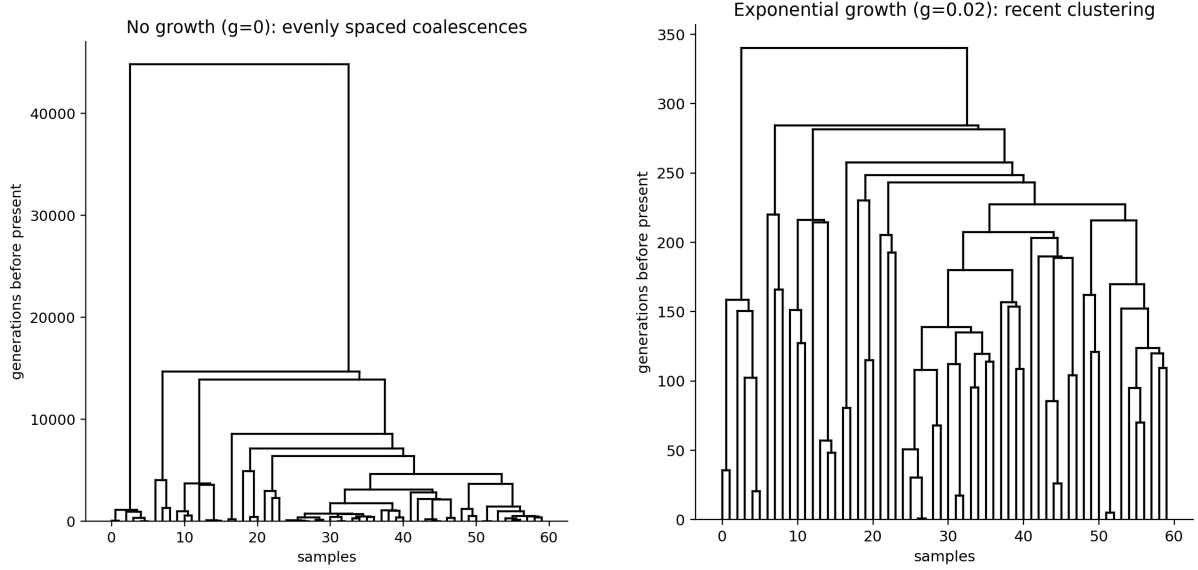
3.1.2 Exponential growth rate g

The parameter g controls how the effective population size changes through time in my simulations. I model a single population whose size changes exponentially backwards in time as

$$N_e(t) = N_{e,0} e^{-gt},$$

where t is time before the present and $N_{e,0} = N_e$ is the present-day effective population size. A positive growth rate $g > 0$ therefore corresponds to population growth forward in time: the population was smaller in the past and has expanded to its current size. When $g = 0$, the population size is constant and the model reduces to the standard constant- N_e Kingman coalescent.

In terms of tree shape, g mainly affects where coalescent events are concentrated in time. With no growth ($g = 0$), coalescent events are spread more evenly across the depth of the tree, giving a roughly balanced distribution of internal node heights. When $g > 0$ (recent expansion), lineages spend a long time in the small ancestral population, so many coalescences occur deeper in the past, while the recent, large population size suppresses very recent coalescences. This produces long external branches and clustering of tips near the present, as illustrated in Figure 11.



(a) No growth ($g = 0$): coalescent events are more evenly spread through time.

(b) Exponential growth ($g = 0.02$): many internal nodes lie deeper in the tree, with long recent branches and clustered tips.

Figure 11: Effect of the exponential growth rate g on coalescent trees, with N_e held fixed. Both panels use the same present-day N_e of 10,000 but different growth rates, highlighting how g shifts the distribution of coalescent times.

3.1.3 Bottleneck timing (t_b) and severity (f_b)

In the 4-parameter model, I add a single bottleneck on top of the exponential-growth background. The bottleneck is described by two parameters:

- t_b : the time before the present (in generations) at which the bottleneck starts;
- $f_b \in (0, 1]$: the severity of the bottleneck, defined as the fraction of the baseline size during the bottleneck, so that the population size becomes

$$N_e^{\text{bottle}} = f_b \cdot N_e.$$

In `msprime`, this is implemented by changing the population size from N_e to $f_b N_e$ at time t_b , and then restoring it back to N_e after a fixed bottleneck duration of 200 generations. Going backwards in time, the population is therefore large (N_e), then spends 200 generations at the reduced size $f_b N_e$, and then returns to the original demographic trajectory.

This temporary drop has a characteristic effect on the genealogy. Because the coalescent rate is inversely proportional to population size, the small population during the bottleneck makes lineages coalesce more rapidly in that time window. As a result, internal nodes tend to *cluster* around t_b , and the strength of this clustering depends on f_b :

- for moderate bottlenecks (e.g. $f_b = 0.5$), the tree shows a visible but mild concentration of coalescent events;
- for severe bottlenecks (e.g. $f_b = 0.05$), many lineages are forced to merge during the bottleneck, producing a much more compressed region in the tree around t_b .

By varying t_b and f_b across simulations, the dataset spans a range of timings and strengths of historical population crashes, allowing the inference methods to be tested on whether they can recover *when* the bottleneck happened and *how strong* it was from tree shape alone.

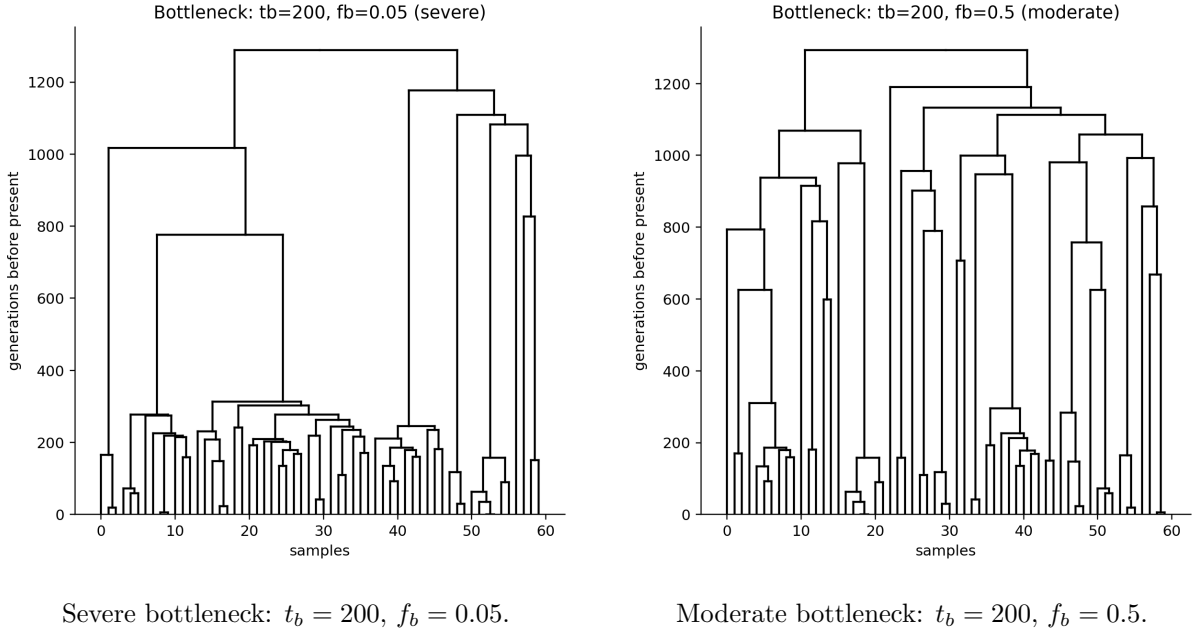


Figure 12: Effect of bottleneck severity on tree shape. Both panels fix the bottleneck time $t_b = 200$ generations and baseline N_e , but vary the severity f_b . A more severe bottleneck (left) forces many coalescences into a narrow time window, while a milder bottleneck (right) produces internal node heights that are more evenly distributed over time heights.

3.2 Representing phylogenies for inference

The raw output of the simulator is a phylogenetic tree: a combinatorial object with a variable number of nodes, a specific branching structure, and continuous branch lengths. While such trees are easy to visualise, they cannot be fed directly into standard neural density estimators, which expect fixed-dimension vector inputs. A central challenge in this thesis is therefore to design a representation of each tree that is (i) low-dimensional and fixed-length, (ii) informative about the underlying demography.

To address this, I consider two complementary approaches:

- **Handcrafted summary statistics.** I compute a set of classical tree-based summary statistics and concatenate them into a single feature vector. This “summary-

statistic embedding” is simple, interpretable, and widely used in Approximate Bayesian Computation.

- **Graph neural network embeddings.** I treat each tree as a graph and learn a representation using a Graph Isomorphism Network (GIN). In an unsupervised setting, the GIN is trained with Deep Graph Infomax (DGI) to produce embeddings that capture generic structural properties of the tree. In a supervised setting, the same GIN backbone is coupled to a regression head and trained directly to predict the demographic parameters from simulated trees.

Both approaches map each phylogeny to a fixed-length vector in $\mathbb{R}^d$, here d is 64, which can then be used as input to the normalising-flow-based Neural Posterior Estimation (NPE) model. In the following sections, I describe these two representation methods in more detail.

3.2.1 Handcrafted summary statistics

To obtain a fixed-length representation of each phylogeny, I first map the tree to a vector of handcrafted summary statistics. These statistics are designed to capture different aspects of tree shape and branch-length structure that are informative about demography.

For a rooted, time-scaled tree T , I use the following summaries:

Total branch length. The sum of the lengths of all branches in the tree. This measures the overall amount of genealogical “history” in the sample and is strongly affected by the effective population size and growth history.

Tree height. The distance in time from the tips to the root (time to the most recent common ancestor). Larger baseline population sizes or earlier bottlenecks tend to increase tree height.

Normalised Colless index. For each internal node, I take the absolute difference in the sizes of its two subtrees and sum these differences over all internal nodes, then normalise by the maximum possible value for a tree with the same number of leaves. This statistic quantifies tree imbalance: perfectly balanced trees have low Colless values, while highly asymmetric trees have high values.

Sackin Sackin’s index is the sum of the depths (number of splits from the root) of all leaves. It reflects how the tree is near the tips: many recent coalescences lead to shallower average depth.

Cherries A cherry is a pair of leaves that share the same immediate parent. The number of cherries, captures fine-scale structure near the present, and is sensitive to recent growth or recent bottlenecks.

Taken together, these statistics cover complementary aspects of the genealogy: total branch length and height summarise the time scale of coalescence; Colless and Sackin indices measure topological balance and depth; and cherries focuses on the recent local structure near the tips. This makes the summary–statistic embedding both interpretable and well matched to the demographic parameters I aim to infer.

Worked example. As a simple illustration, consider the rooted tree in Figure 13, with branch lengths labelled on each edge.

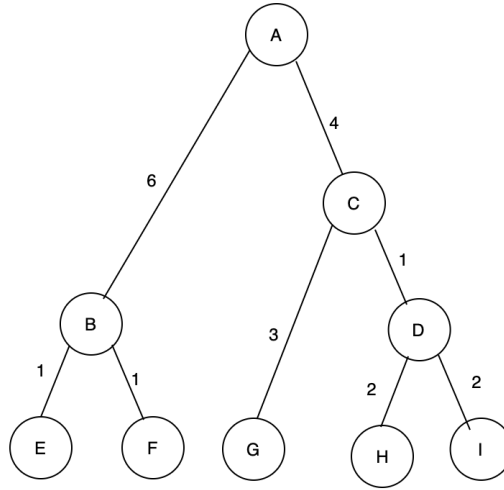


Figure 13: Rooted tree used to illustrate the handcrafted summary statistics.

The tree has five leaves (E, F, G, H, I) and four internal nodes (A, B, C, D). Using the definitions above:

- **Total branch length.**

$$L_{\text{tot}} = 6 + 4 + 1 + 1 + 3 + 1 + 2 + 2 = 20.$$

- **Tree height.** All root-to-leaf paths have length

$$d(A, E) = d(A, F) = d(A, G) = d(A, H) = d(A, I) = 7,$$

so the tree height is $h = 7$.

- **Normalised Colless index.** For each internal node, let n_L and n_R be the numbers of leaves in its two subtrees. The absolute differences are

$$|n_L - n_R|_B = 0, |n_L - n_R|_D = 0, |n_L - n_R|_C = |1 - 2| = 1, |n_L - n_R|_A = |2 - 3| = 1,$$

giving Colless index $C = 2$. For a binary tree with $n = 5$ leaves, the maximum possible value is $C_{\max} = \frac{(n-1)(n-2)}{2} = 6$, so the normalised Colless index is

$$C_{\text{norm}} = \frac{C}{C_{\max}} = \frac{2}{6} \approx 0.33.$$

- **Sackin** Leaf depths (number of edges from the root) are

$$\text{depth}(E) = \text{depth}(F) = \text{depth}(G) = 2, \quad \text{depth}(H) = \text{depth}(I) = 3.$$

Sackin's index is

$$S = 2 + 2 + 2 + 3 + 3 = 12,$$

- **Cherries** There are two cherries: (E, F) under B and (H, I) under D , so

$$\text{cherries} = 2.$$

For this tree, the handcrafted summary-statistic embedding is therefore the vector

$$(L_{\text{tot}}, h, C_{\text{norm}}, S, \text{cherries}) = (20, 7, 0.33, 12, 2).$$

Note. Throughout this thesis, only the Colless index is normalised (we rescale C by its maximum possible value for a tree with n tips); the Sackin index and number of cherries are used in their raw, unnormalised form. In all simulations, I fix the sample size to 30 (so there are 60 tips in the tree) and set the recombination rate to zero, so using total versus mean versions of branch-length-based statistics does not change the relative comparison between trees, since they differ only by a constant scaling factor.

Limitations of handcrafted summary statistics While summary statistics provide a simple and interpretable way to represent trees, they also introduce important limitations. First, the mapping from a full tree to a short vector is highly lossy: many different tree topologies and branch-length patterns can produce exactly the same set of statistics. As a result, subtle but potentially informative aspects of the genealogy may not be encoded.

Second, the representation is hand-designed: I must choose in advance which statistics to include, which inevitably biases the representation toward patterns that are already known or suspected to be important. Finally, all trees are forced into the same fixed set of global numbers, which makes it difficult to capture richer structure that might become relevant for more complex demographic models.

These drawbacks motivate the use of representation-learning methods that operate directly on the tree as a graph and can, in principle, learn which structural features are

informative for the inference task.

3.2.2 Graph neural network embeddings

To move beyond handcrafted features, I represent each phylogeny as a graph and learn its embedding using a Graph Isomorphism Network (GIN). In this view, the nodes of the tree correspond to sampled tips and internal ancestral nodes; edges correspond to parent–child relationships; and node features encode simple local information. A GIN is a message-passing graph neural network that repeatedly updates each node’s hidden state by aggregating information from its neighbours and applying a small multilayer perceptron (MLP).

To apply a GIN to phylogenies, I first represent each simulated tree as a graph and attach a small feature vector to every node. These node features encode both temporal information (where the node lies in time) and structural information (how it sits inside the tree). For each node I use the following *raw* features:

- **time**: time of the node measured from the present (e.g. generations before present).
Rationale: coalescent times are directly shaped by the demographic model; knowing when an ancestral node occurred is crucial for detecting growth and bottlenecks.
- **depth**: the number of edges on the path from the root to the node (a purely topological depth).
Rationale: separates nodes that are near the root from those near the tips.
- **subtree_size**: the number of descendant leaves (tips) below the node.
Rationale: measures how large the clade represented by the node is, and may help the GIN detect imbalance patterns and distinguish large from small subtrees.
- **is_leaf**: a binary indicator (1 for tips, 0 for internal nodes).
Rationale: tips and internal nodes play different roles in the genealogy.
- **parent_branch_len**: length of the edge from the node to its parent.
Rationale: captures how much time passes between successive coalescent events.

Because these raw quantities can vary widely across trees (for example, trees with very different overall heights), I also include normalised versions that express relative position within each tree:

- **time_norm** = $\text{time}/t_{\text{root}}$: node time scaled by the height of the tree.
Rationale: makes node times comparable across trees of very different absolute height; values close to 1 correspond to nodes near the root, values close to 0 to nodes near the present.

- **depth_norm** = $\text{depth}/d_{\max}$: depth scaled by the maximum depth in the tree.
Rationale: emphasises relative topological position (top vs. mid vs. bottom of the tree) without being sensitive to the exact number of levels.
- **subtree_frac** = $\text{subtree_size}/n_{\text{samples}}$: fraction of all tips that lie below the node.
Rationale: allows the network to reason about what proportion of the sample belongs to a given clade.
- **bl_to_root_norm** = $(\text{branch length from root to node})/L_{\text{tot}}$: cumulative branch length along the path from the root to the node, divided by the total branch length of the tree.
Rationale: encodes how far along the genealogy (in terms of elapsed evolutionary time) a node sits, relative to the whole tree.

Together, these features give the GIN a compact but informative view of each node’s *when* (time), *where* (depth and distance to the root), and *how large a clade* it represents (subtree size / fraction), providing a rich basis for learning tree embeddings directly from the graph structure.

3.2.3 Message passing in the Graph Isomorphism Network

The Graph Isomorphism Network (GIN) operates directly on the tree viewed as a graph $G = (V, E)$, with one feature vector x_v for each node $v \in V$ (constructed from the node features described above). These features form the initial node embeddings:

$$h_v^{(0)} = x_v \quad \text{for all } v \in V.$$

Each GIN layer then performs message passing in two steps, illustrated in Figures 14 and 15.

Neighbour aggregation. For node v at layer k , the layer first aggregates information from its neighbours:

$$m_v^{(k)} = (1 + \epsilon^{(k)}) h_v^{(k-1)} + \sum_{u \in \mathcal{N}(v)} h_u^{(k-1)}, \quad (1)$$

where $\mathcal{N}(v)$ is the set of neighbours of v , $h_v^{(k-1)}$ is the previous-layer embedding, and $\epsilon^{(k)}$ is a learnable scalar parameter.

Non-linear update via MLP. The aggregated message is then passed through a small multilayer perceptron (MLP) to obtain the new node embedding:

$$h_v^{(k)} = \text{MLP}_k(m_v^{(k)}). \quad (2)$$

In my implementation, the GIN encoder consists of two such layers, each with a two-layer MLP and batch normalisation. After these message-passing steps, every node v has a final representation $h_v^{(2)} \in \mathbb{R}^{64}$. To obtain a single embedding for each tree, I sum the node embeddings within that tree (using the batch vector to separate trees), giving a graph-level vector $g \in \mathbb{R}^{64}$. A final linear layer maps this to an output embedding $z \in \mathbb{R}^{64}$, which I use as the learned representation of the phylogeny.

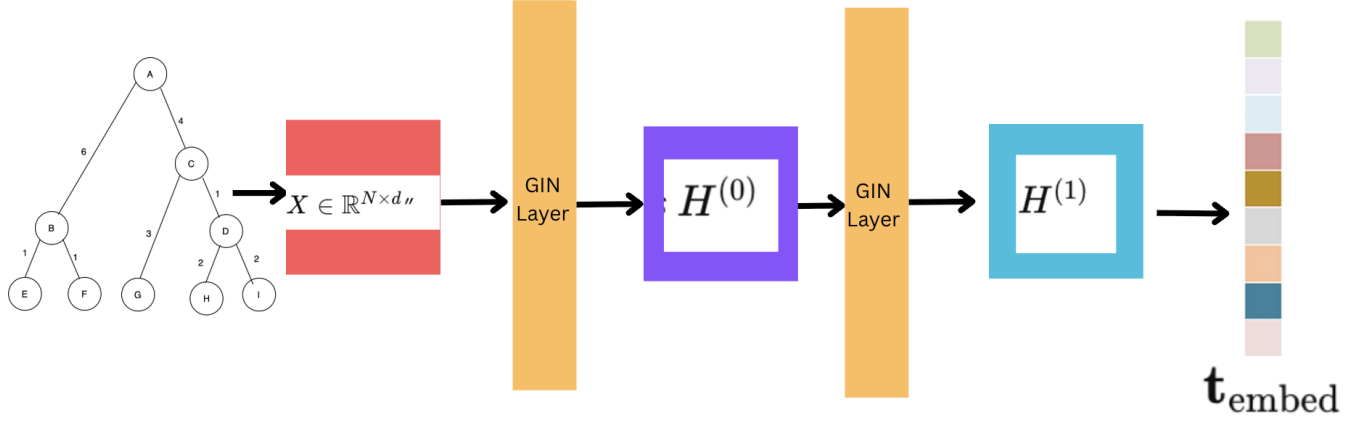


Figure 14: High-level view of the GIN encoder. The tree is first encoded as a node-feature matrix $X \in \mathbb{R}^{N \times d_{in}}$, then passed through two GIN layers producing node embeddings $H^{(0)}$ and $H^{(1)}$. A sum over nodes followed by a linear layer yields the final tree embedding z .

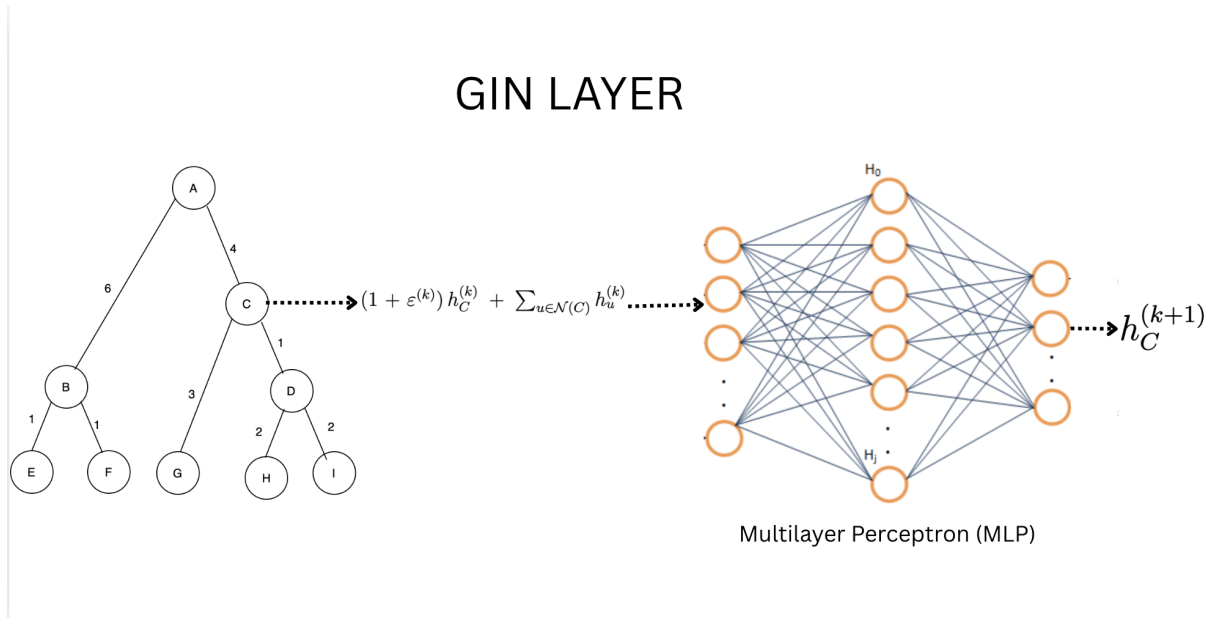


Figure 15: Illustration of a single GIN layer update for node C . The node collects its own embedding and those of its neighbours, forming the aggregated message $(1 + \epsilon^{(k)})h_C^{(k)} + \sum_{u \in \mathcal{N}(C)} h_u^{(k)}$, which is then passed through an MLP to produce the updated embedding $h_C^{(k+1)}$.

Unsupervised training: plain DGI In the unsupervised setting, I train the GIN encoder using Deep Graph Infomax (DGI), whose goal is to learn node- and graph-level embeddings that capture as much information as possible about the true graph structure, without using labels. The architecture is illustrated in Figure 16, which builds on the GIN encoder shown in Figure 14.

Using the notation in Figure 16:

- G : original tree (graph) from the simulator.
- C : corruption function that produces a “fake” graph $H = C(G)$ (in practice I keep the edges but randomly shuffle node features).
- E : GIN encoder (message passing) that maps node features to node embeddings.
- R : readout function that aggregates node embeddings of the real graph into a single summary vector s (I use SUM pooling followed by a linear layer).
- D : discriminator, which scores pairs (node embedding, s).

Training proceeds as follows:

1. **Encode the real graph.** Pass G through the GIN encoder E to obtain node embeddings $\{h_i\}_{i=1}^N$.
2. **Compute a global summary.** Apply the readout R to the node embeddings to obtain a graph-level summary vector

$$s = R(\{h_i\}_{i=1}^N).$$

3. **Encode the corrupted graph.** Apply the same encoder E to the corrupted graph $H = C(G)$, giving corrupted node embeddings $\{\tilde{h}_i\}_{i=1}^N$.
4. **Score real vs. corrupted pairs.** For each node i , the discriminator D produces a scalar score for the real and corrupted pairs:

$$D(h_i, s), \quad D(\tilde{h}_i, s),$$

where high scores should correspond to real pairs (h_i, s) and low scores to corrupted pairs $(\tilde{h}_i, s)$. In practice, $D(\cdot, \cdot)$ is a small bilinear/MLP followed by a sigmoid.

The DGI loss (Yin et al. 2023) encourages high scores for real pairs and low scores for corrupted pairs. For N nodes it is

$$\mathcal{L}_{\text{DGI}} = -\frac{1}{N} \sum_{i=1}^N \left[\log \sigma(D(h_i, s)) + \log (1 - \sigma(D(\tilde{h}_i, s))) \right].$$

Minimising $\mathcal{L}_{\text{DGI}}$ is equivalent to maximising a mutual-information lower bound between local node embeddings and the global summary of the true graph. With this objective alone, the GIN learns embeddings that capture generic structural and temporal patterns in the phylogenies, without ever seeing the true demographic parameters.

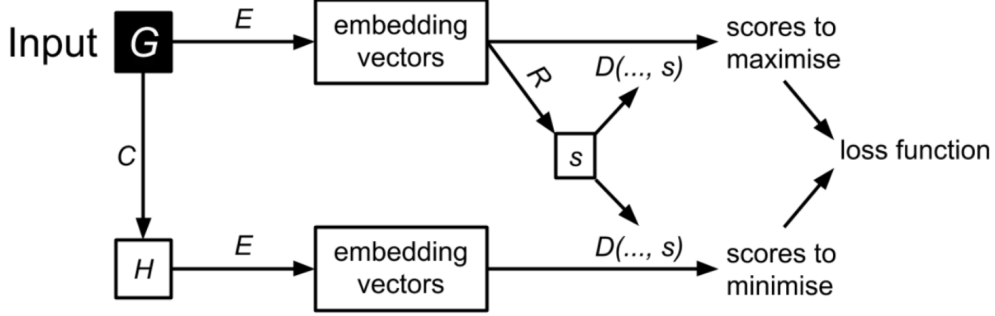


Figure 16: Unsupervised training of the GIN encoder with Deep Graph Infomax. The original graph G and a corrupted version $H = C(G)$ are both encoded by the same GIN encoder E . A readout R produces a global summary vector s for the real graph. The discriminator D assigns high scores to real pairs (h_i, s) and low scores to corrupted pairs $(\tilde{h}_i, s)$; these scores are combined into the DGI loss.

Source: Taken from Wilson (2020).

Adding supervision: predicting the parameters For the supervised variant, I keep the same GIN encoder and DGI, but additionally use the graph embedding to predict the demographic parameters that generated each tree.

From the node embeddings of the real graph G , I obtain a graph-level embedding

$$z_G = \text{graph_embed}(\{h_i\}_{i=1}^N),$$

using the SUM readout and linear layer of the `GINEncoder` (Section 3.2.2). On top of z_G I add a small regression head that outputs

$$\hat{\theta} = (\hat{N}_e, \hat{g}, \hat{t}_b, \hat{f}_b),$$

an estimate of the true simulation parameters $\theta = (N_e, g, t_b, f_b)$.

The supervised loss is a mean squared error (MSE)

$$\mathcal{L}_{\text{sup}} = \frac{1}{4} \sum_{d=1}^4 (\hat{\theta}_d - \theta_d)^2.$$

The total training objective combines unsupervised DGI and supervised regression:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{DGI}} + \lambda \mathcal{L}_{\text{sup}},$$

with λ controlling how strongly the supervised signal is weighted (I use $\lambda = 0.1$).

Under this combined objective,

- the DGI term still pushes the encoder to learn globally consistent embeddings that capture structural information about the tree; and
- the supervised term further shapes these embeddings so that they are specifically useful for recovering the demographic parameters (N_e, g, t_b, f_b) .

It is important to emphasise that this supervised prediction task is *not* the final inference model that produces the posterior distribution over parameters. The regression head is used only during representation learning to guide the GIN encoder towards embeddings that are informative about (N_e, g, t_b, f_b) . Once training is complete, I discard this auxiliary head and use the learned graph embeddings as inputs to the separate Neural Posterior Estimation (NPE) model, which is the component that actually learns the posterior distribution over demographic parameters.

3.3 Neural Posterior Estimation (NPE)

Once each simulated tree has been turned into a fixed-length vector (either by summary statistics or by a GIN-based embedding), the next step is to learn how those vectors relate to the underlying demographic parameters $\theta = (N_e, g, t_b, f_b)$. For this I use Neural Posterior Estimation (NPE), a simulation-based inference method based on conditional normalising flows.

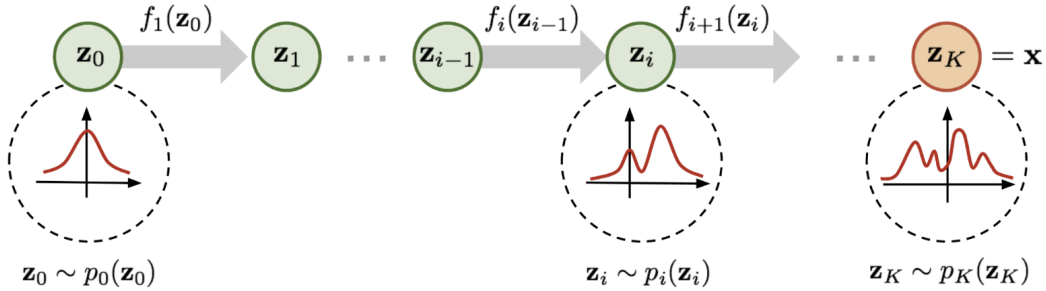


Figure 17: Schematic view of the normalising flow used in NPE. A simple base variable $z_0 \sim p_0(z_0)$ (e.g. standard Gaussian) is transformed through a sequence of invertible neural networks $f_1, \dots, f_K$ to produce z_K , which follows a flexible target density $p_K(z_K)$. In NPE, this flow is conditioned on the tree representation so that the final density approximates the posterior $p(\theta \mid \text{tree})$.

Source: Taken from Weng (2018).

Normalising-flow model. Practically, I use the `sbi` library’s NPE implementation (SBI Developers 2025) with a Neural Spline Flow (NSF) as the density estimator. For a given tree representation x :

- A base variable $z_0 \sim p_0(z_0)$ (typically a standard multivariate Gaussian) is sampled.
- A sequence of invertible, x -conditioned transformations $f_1, \dots, f_K$ maps

$$z_K = f_K \circ \dots \circ f_1(z_0; x),$$

inducing a flexible density $p_K(z_K \mid x)$.

- The flow is trained so that the resulting density over z_K matches the simulated parameters θ associated with similar representations x .

In other words, the model learns:

$$x \longmapsto q_\phi(\theta \mid x) \approx p(\theta \mid x),$$

where q_ϕ is the normalising flow with parameters ϕ .

Amortised inference. A key advantage is that NPE is *amortised*: once the flow has been trained on a large bank of simulations, inference for new trees is computationally cheap:

1. Compute the representation x_{obs} of the observed tree.
2. Feed x_{obs} into the trained NPE model.
3. Draw many posterior samples $\theta^{(1)}, \dots, \theta^{(S)} \sim q_\phi(\theta \mid x_{\text{obs}})$ and compute desired posterior quantities.

No additional simulations are needed at test time; all expensive computation happens once during training.

Role of summary statistics vs. GNN embeddings. From the NPE model’s perspective, there is no algorithmic difference between the two representation strategies:

- with summary statistics, the input x is a hand-crafted feature vector;
- with GIN embeddings, the input x is a learned feature vector produced by the GNN encoder.

In both cases, NPE simply receives a vector $x \in \mathbb{R}^d$ and learns a conditional posterior over θ . This makes NPE a clean framework for comparing representations: any difference in inference quality between the two approaches can be attributed to how informative their embeddings are, rather than to changes in the posterior-inference method itself.

3.4 Training vs. testing

The overall workflow separates naturally into a *training phase*, where the models learn from simulated data with known parameters, and a *testing phase*, where only trees are observed and the goal is to infer their underlying demography. During training we have access to both

- the simulated parameters $\theta = (N_e, g, t_b, f_b)$, and
- the corresponding tree generated from θ .

These two pieces of information are used in different ways by the embedding models and by the posterior estimator.

Training phase.

Embedding learning

- **Summary statistics.** There is no learning step: the embedding is a deterministic function of the tree. For each simulated tree I directly compute the summary-statistic vector.
- **Unsupervised GIN (DGI).** The encoder sees only the tree. Using Deep Graph Infomax, the GIN is trained to discriminate between real graphs and corrupted versions of the same graphs. The true parameters θ are *never* used in this objective; the embeddings are driven purely by structural and temporal patterns in the trees.
- **Supervised GIN (DGI + regression).** Here the encoder uses both the tree and its true parameters. In addition to the unsupervised DGI loss, I attach a regression head that takes the graph embedding and predicts $\hat{\theta} = (\hat{N}_e, \hat{g}, \hat{t}_b, \hat{f}_b)$. A mean-squared-error loss between $\hat{\theta}$ and the true θ encourages the embeddings to be explicitly informative about the demographic parameters.

Posterior model (NPE)

For all three representations, the Neural Posterior Estimation model is trained on many *labelled* pairs

$$(\text{embedding}, \theta),$$

where the embedding is obtained from the tree (summary statistics, unsupervised GIN, or supervised GIN). NPE learns a conditional density $q_\phi(\theta | x)$ so that, after training, it can map an embedding x to a full posterior distribution over θ .

Testing phase (inference). In the testing phase we no longer know θ ; we only observe a new tree (currently its simulated but in future we could use a real data). The inference pipeline is:

1. **Convert the tree to an embedding.** Compute the appropriate representation (summary-statistic vector, unsupervised GIN embedding, or supervised GIN embedding using the trained encoder).
2. **Feed the embedding into NPE.** Pass this vector into the trained NPE model.
3. **Obtain the posterior over parameters.** NPE outputs the posterior distribution $q_\phi(\theta \mid x_{\text{obs}})$ for that tree, from which I can draw samples.

Importantly, during testing no true labels are used anywhere: the supervised GIN regression head is only active during training, and NPE is evaluated purely in its inference mode. All supervision (the parameter labels θ) is confined to the training phase, while the testing phase operates exactly as it would on real data where the true demography is unknown.

4 PhyloInference Dashboard

Alongside the simulation and inference pipeline, I implemented an interactive web dashboard, *PhyloInference Play*, to explore the models in real time. The dashboard lets the user specify demographic parameters (such as N_e , g , t_b , and f_b), simulate a tree under the coalescent model, and then visualise both the genealogy and the corresponding inference outputs.

The interface supports several views:

- **Tree and lineage plots:** visualisation of the simulated phylogeny and the lineage-through-time (LTT) curve.
- **Posterior** panels that display posterior marginal distributions from NPE for different parameter settings.
- **GNN–NPE view:** inspection of posteriors obtained using the GNN-based embeddings.
- **Bookmarking and notes:** the ability to save particular parameter configurations, revisit them later and attach short textual notes.

This dashboard was primarily a research tool: it allowed rapid iteration over different parameter combinations, visual inspection of the resulting trees, and direct comparison

between the inference methods, making it easier to understand when and why particular representations succeed or fail.

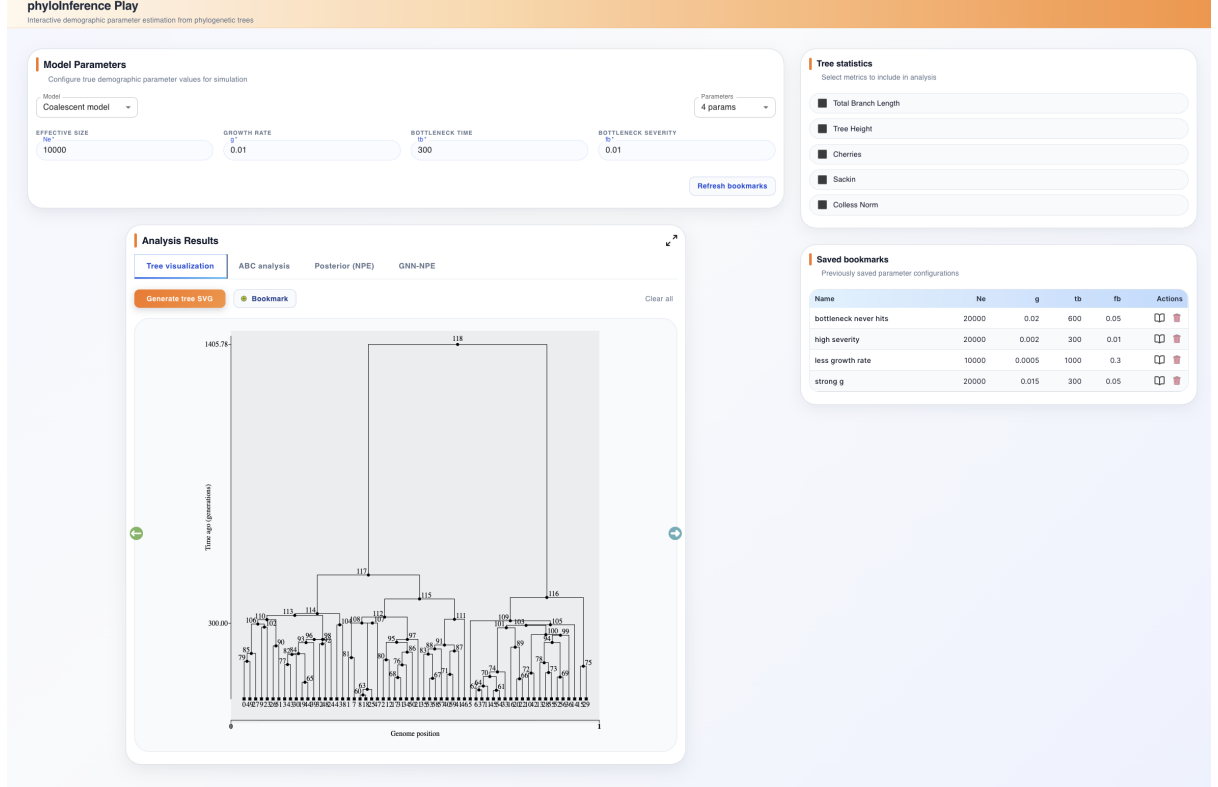


Figure 18: Screenshot of the *PhyloInference Play* dashboard. The user can set demographic parameters, generate and visualise trees, inspect ABC and NPE posteriors, and bookmark parameter configurations with notes.

5 Evaluation

The goal of the evaluation is to compare how well different tree representations hand-crafted summary statistics, unsupervised GIN, and supervised GIN support posterior inference of the demographic parameters $\theta = (N_e, g, t_b, f_b)$ from a single observed phylogeny. For each parameter setting and each representation, I report both *parameter-space errors* and *summary-statistic errors*, together with qualitative inspection of the posterior marginals.

5.1 Parameter-space errors

Let $\theta_{\text{true}} \in \mathbb{R}^4$ denote the true parameter vector for a given tree and $\hat{\theta} \in \mathbb{R}^4$ a point estimate extracted from the posterior. For each component $d \in \{1, 2, 3, 4\}$, the absolute parameter error is

$$\text{APE}_d = |\hat{\theta}_d - (\theta_{\text{true}})_d|.$$

In our experiments we use a maximum a posteriori (MAP) estimate as the point estimate. Given an observed tree representation z , the MAP estimate is defined as

$$\hat{\theta}_{\text{MAP}}(z) \in \arg \max_{\theta} p(\theta | z),$$

i.e. the parameter vector at which the learned posterior density attains its maximum. Practically, we obtain $\hat{\theta}$ using the `sbi` library (SBI Developers 2025), which provides a *MAP utility* (`posterior.map(...)`) that returns a high-density sample from the trained posterior for a given observation.

To obtain a single scalar that summarises how far the estimate is from the truth in parameter space, I also use the ℓ_1 norm of the absolute errors,

$$E_{\text{param}}^{\ell_1} = \sum_d \Delta \theta_d^{\text{abs}} = \sum_d |\hat{\theta}_d - \theta_{\text{true},d}|.$$

This is reported in the tables as $\sum_j |\Delta \theta_j|$. Absolute errors, however, depend on the units and magnitudes of each parameter. To compare errors across parameters with different scales, I also report *relative parameter errors*. For each parameter with non-zero true value, I define

$$\Delta \theta_d^{\text{rel}} = \frac{|\hat{\theta}_d - \theta_{\text{true},d}|}{|\theta_{\text{true},d}|},$$

and express this as a percentage in the tables. For parameters whose true value is exactly zero (such as g in the baseline setting), this quantity is undefined, and I indicate it with “—” rather than forcing a numerical value.

To summarise performance across parameters, I also report a *mean relative parameter error* over a subset of components, for example

$$\bar{r}_{\text{param}} = \frac{1}{3} \left(\Delta \theta_{N_e}^{\text{rel}} + \Delta \theta_{t_b}^{\text{rel}} + \Delta \theta_{f_b}^{\text{rel}} \right),$$

which averages the relative errors for N_e , t_b , and f_b (omitting g when its true value is zero). This gives a single, scale-free number that can be compared between representations and parameter settings.

5.2 Summary-statistic errors

Although the ultimate goal is to recover the demographic parameters, it is also informative to measure how well the models reproduce the observed tree statistics. For each tree I have a vector of handcrafted summary statistics

$$s_{\text{true}} = (\text{TBL}, \text{height}, \text{cherries}, \text{Sackin}, \text{Colless}_{\text{norm}}),$$

and, for each representation, a corresponding set of statistics s_{pred} obtained by simulating trees from the inferred parameters and recomputing the same statistics.

For each statistic k I report the *absolute summary-statistic error*

$$\Delta s_k^{\text{abs}} = |s_{\text{pred},k} - s_{\text{true},k}|,$$

and an aggregate ℓ_1 error

$$E_{\text{stats}}^{\ell_1} = \sum_k \Delta s_k^{\text{abs}},$$

listed in the tables as $\sum_k |\Delta s_k|$.

As with the parameters, absolute errors depend on the scale of each statistic. I therefore also report *relative summary-statistic errors*,

$$\Delta s_k^{\text{rel}} = \frac{|s_{\text{pred},k} - s_{\text{true},k}|}{|s_{\text{true},k}|},$$

again shown as percentages. These allow a fair comparison between statistics with very different magnitudes (for example, total branch length versus Colless index). I also summarise these into a mean relative statistic error

$$\bar{r}_{\text{stats}} = \frac{1}{K} \sum_{k=1}^K \Delta s_k^{\text{rel}},$$

which provides a single, scale-free measure of how closely the inferred trees resemble the original tree in terms of the chosen summary statistics.

Reporting both parameter-space and statistic-space errors is useful because a representation might match the summary statistics well while still mis-estimating the underlying parameters (or vice versa). Together, these metrics help diagnose which aspects of the genealogy each representation captures or misses.

5.3 Qualitative inspection of posterior distributions

In addition to scalar error metrics, I also inspect the full posterior marginal distributions for each parameter, as shown in the NPE plots (6) for each parameter setting and representation. For each marginal, this qualitative checks complement the numerical errors: two models may have similar point-estimate errors but very different posterior shapes (e.g. one overly confident but biased, another more dispersed but better-calibrated). By combining posterior plots with the absolute and relative errors defined above, I obtain a more complete picture of how each representation supports Bayesian inference for the demographic parameters.

6 Results

In this section we evaluate how well the different representations (summary statistics, unsupervised GIN, and supervised GIN) recover the parameters (N_e, g, t_b, f_b) from a single observed phylogeny. For clarity, I present one subsection per parameter setting.

6.1 Parameter setting 1: Moderate size, early bottleneck

True parameters and motivation

As a first test case we use a simple coalescent bottleneck scenario with moderate effective size, no growth, and an early, strong bottleneck:

$$\theta_{\text{true}} = (N_e, g, t_b, f_b) = (5000, 0, 1000, 0.1).$$

This setting produces a clearly distorted tree while remaining close to the centre of the prior ranges, making it a convenient reference point for comparing the three representations.

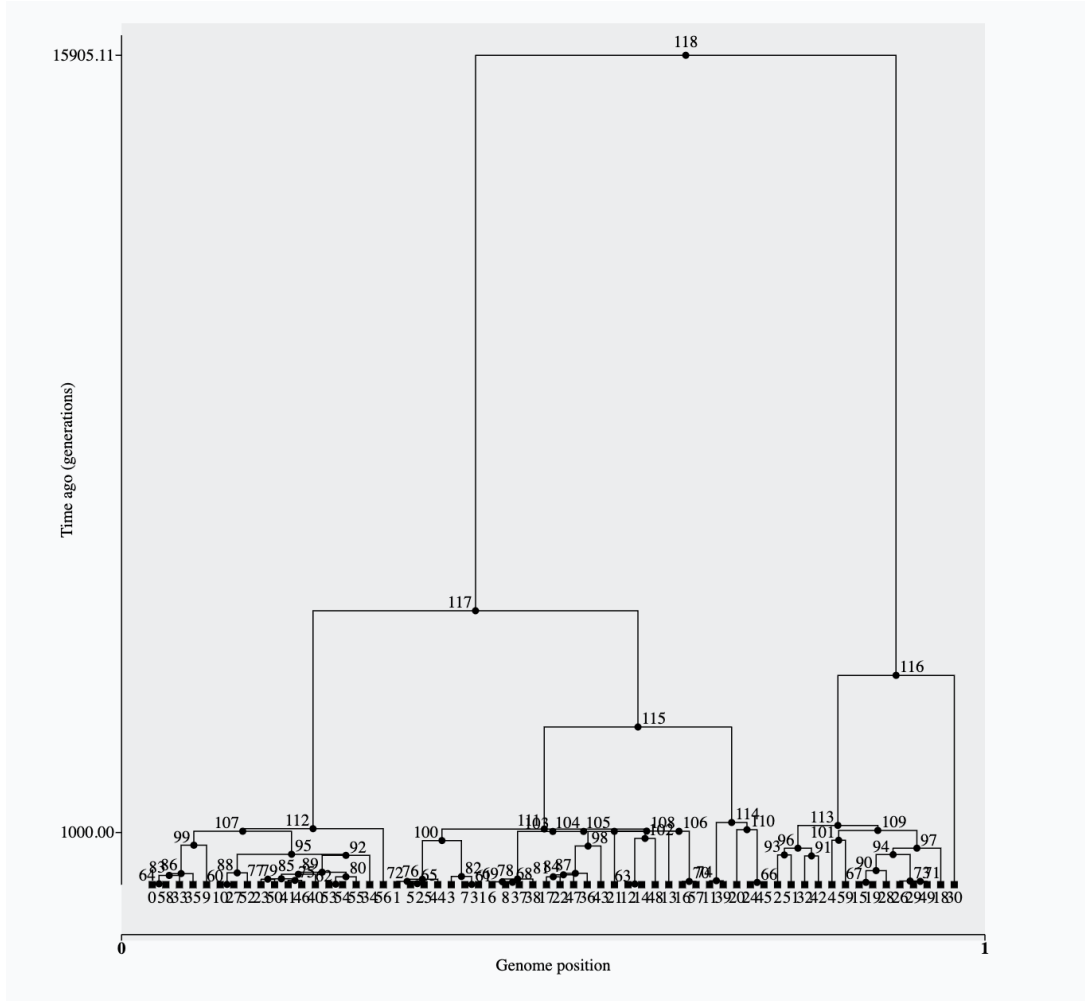


Figure 19: Phylogeny simulated at $\theta_{\text{true}} = (5000, 0, 1000, 0.1)$.

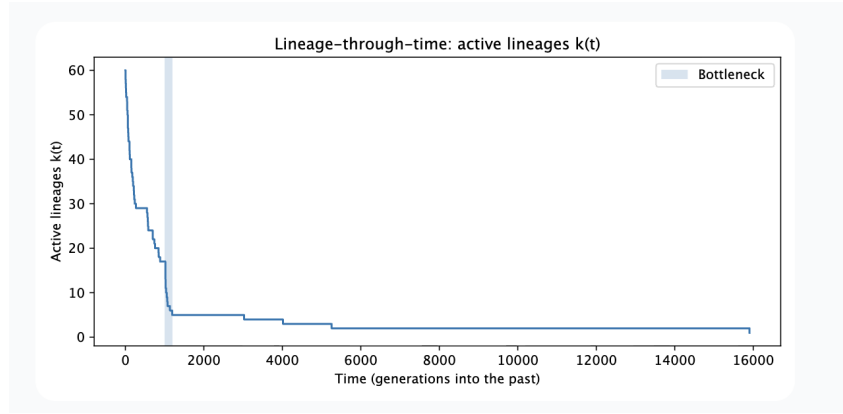


Figure 20: Lineage-through-time plot for the tree in Figure 19.

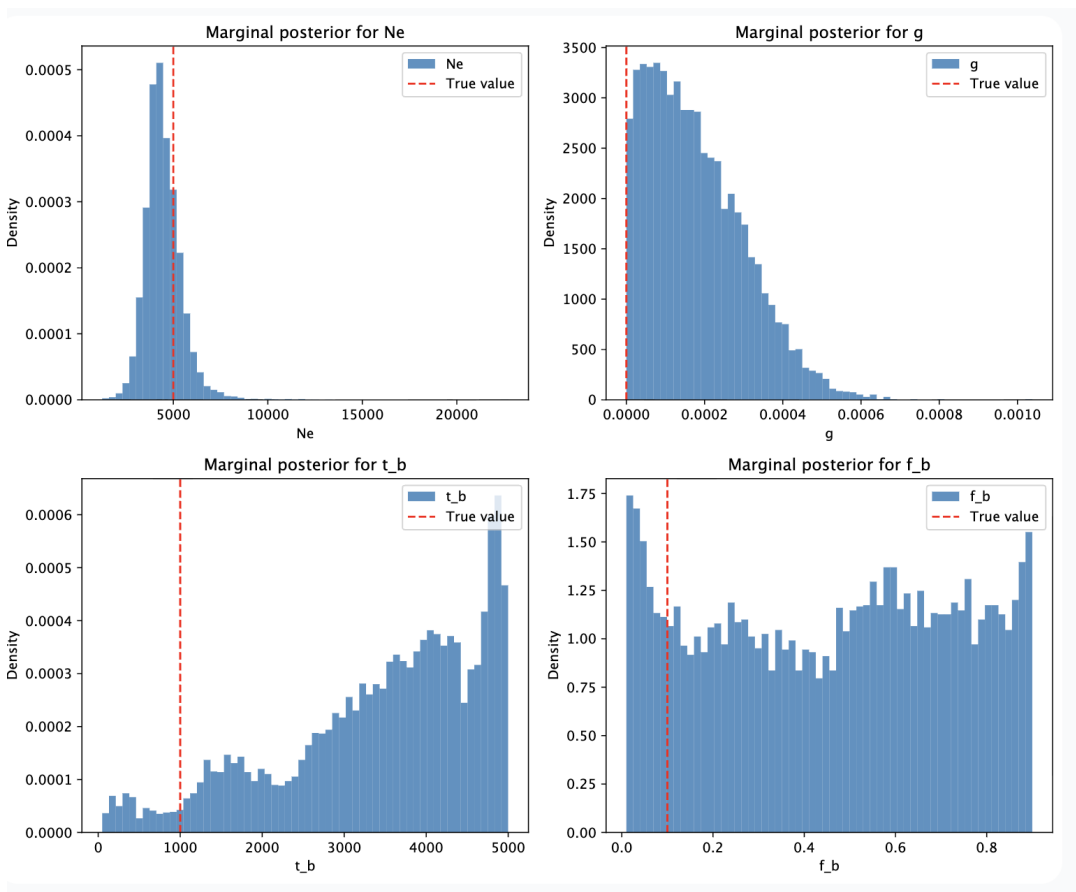


Figure 21: Posterior marginal distributions for (N_e, g, t_b, f_b) using the summary-statistics embedding.

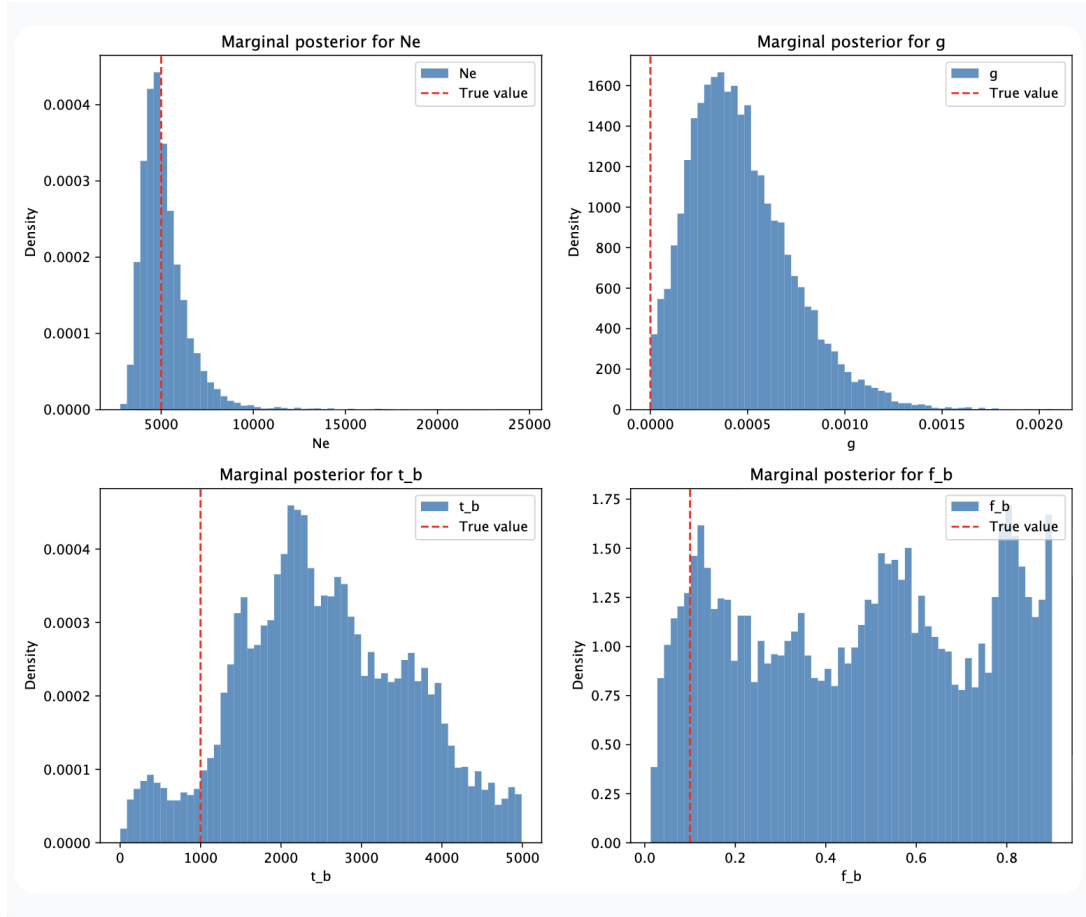


Figure 22: Posterior marginals using the unsupervised GIN embedding.

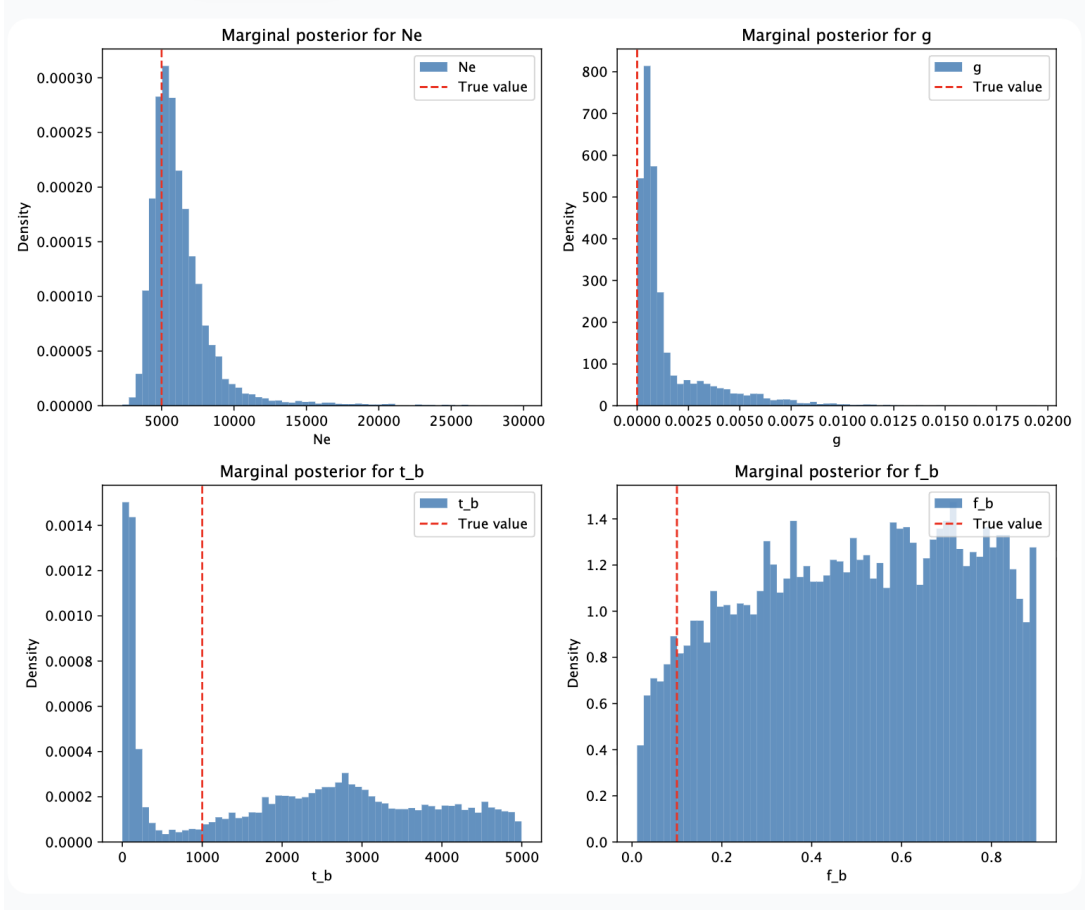


Figure 23: Posterior marginals using the supervised GIN embedding.

Table 3: Summary statistics for the tree in Figure 19.

	total branch length	tree height	cherries	Sackin	Colless _{norm}
True value	89276.148	24970.723	22	425	0.074

Table 4: Absolute parameter errors for Parameter setting 1.

Model	$ \Delta N_e $	$ \Delta g $	$ \Delta t_b $	$ \Delta f_b $	$\sum_j \Delta \theta_j $
Summary stats	142.241	8.84×10^{-6}	1948.548	0.079	2090.869
GNN (unsup.)	1120.857	7.84×10^{-5}	1889.365	0.021	3010.243
GNN (sup.)	936.072	8.13×10^{-4}	1164.176	0.685	2100.933

Table 5: Absolute errors in summary statistics for Parameter setting 1.

Model	$ \Delta \text{TBL} $	$ \Delta \text{Height} $	$ \Delta \text{cherries} $	$ \Delta \text{Sackin} $	$ \Delta \text{Colless}_{\text{norm}} $	$\sum_k \Delta s_k $
Summary stats	38982.277	21911.627	6	75	0.058	60974.962
GNN (unsup.)	37765.391	15140.096	1	3	0.010	52909.496
GNN (sup.)	35540.238	18041.483	1	34	0.019	53616.740

Table 6: Relative parameter errors (percent) for Parameter setting 1.

Model	$ \Delta N_e /N_e$ [%]	$ \Delta g / g $ [%]	$ \Delta t_b /t_b$ [%]	$ \Delta f_b /f_b$ [%]
Summary stats	2.8	–	194.9	79.0
GNN (unsup.)	22.4	–	188.9	20.8
GNN (sup.)	18.7	–	116.4	684.8

Table 7: Relative errors (percent) in summary statistics for Parameter setting 1.

Model	TBL [%]	Height [%]	Cherries [%]	Sackin [%]	Colless _{norm} [%]
Summary stats	43.7	87.7	27.3	17.6	78.0
GNN (unsup.)	42.3	60.6	4.5	0.7	13.4
GNN (sup.)	39.8	72.3	4.5	8.0	25.2

For reference, the mean relative parameter error (averaged over N_e , t_b , and f_b) is

$$\bar{r}_{\text{param}} = 0.92 \text{ (summary stats), } 0.77 \text{ (GNN unsup.), } 2.73 \text{ (GNN sup.),}$$

and the mean relative summary-statistic error is

$$\bar{r}_{\text{stats}} = 0.51 \text{ (summary stats), } 0.24 \text{ (GNN unsup.), } 0.30 \text{ (GNN sup.).}$$

Results. In this moderate early bottleneck scenario, the summary-statistics representation recovers N_e and g very accurately but places most posterior mass for t_b far from the true value, and also underestimates

6.2 Parameter setting 2: Recent severe bottleneck with growth

True parameters and rationale for this choice

For the second configuration I consider a scenario with a larger effective population size and both recent growth and a very severe bottleneck:

$$\theta_{\text{true}} = (N_e, g, t_b, f_b) = (10,000, 0.01, 300, 0.01).$$

Compared to the baseline in Section 6.1, this setting has (i) a larger N_e , (ii) positive exponential growth ($g = 0.01$), (iii) a more recent bottleneck ($t_b = 300$), and (iv) a much stronger severity ($f_b = 0.01$). This combination produces a relatively shallow tree with short total branch length, and is intended to test whether the representations can recover parameters when strong recent events dominate the genealogy.

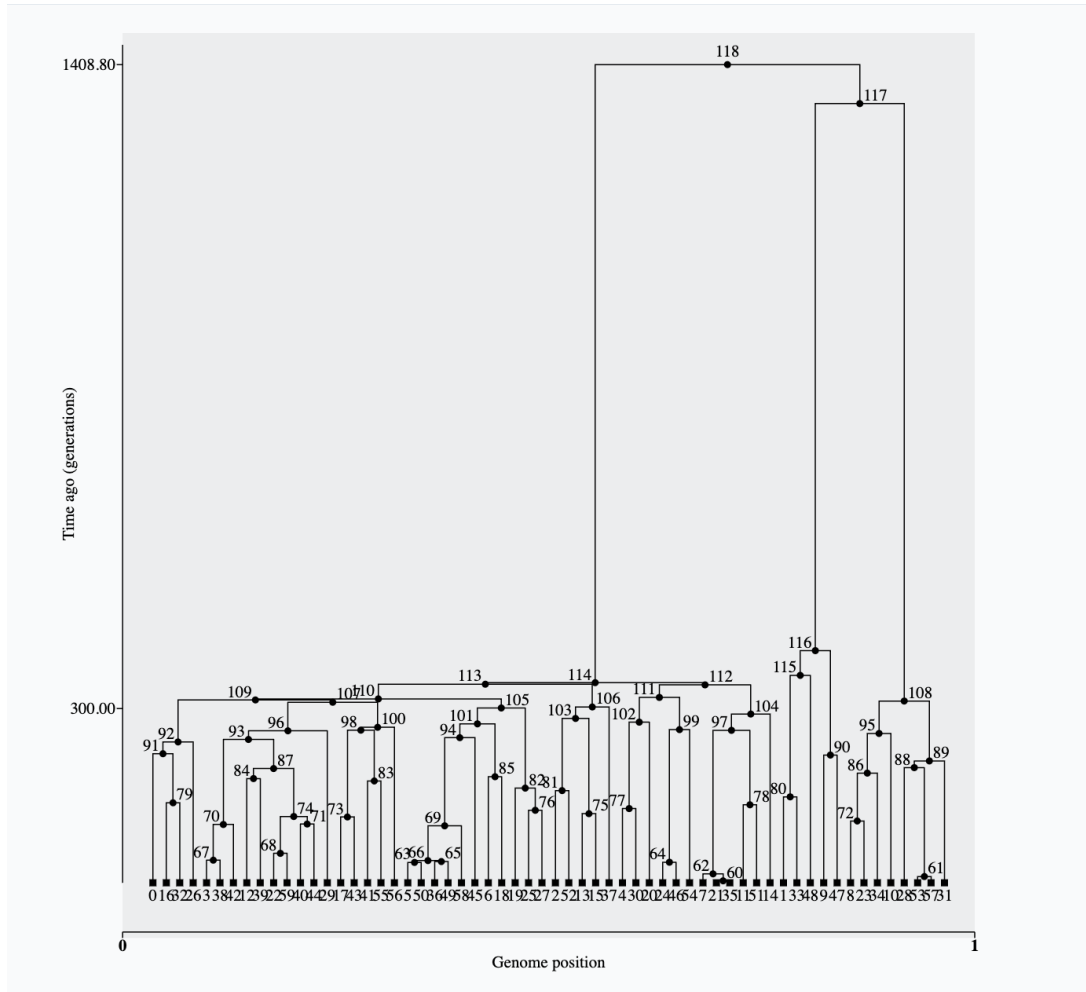


Figure 24: True phylogeny corresponding to Parameter setting 2.

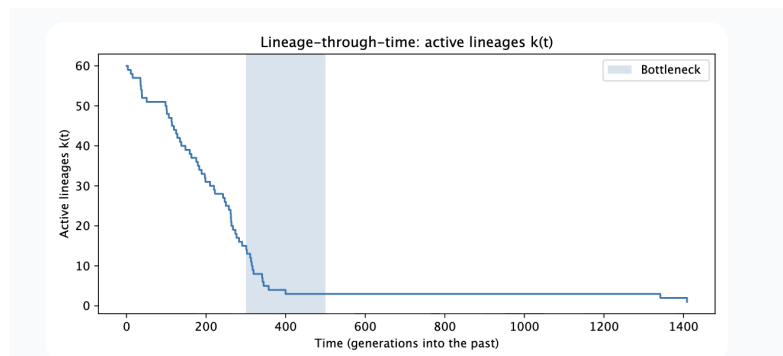


Figure 25: Lineage-through-time (LTT) plot for the same tree as in Figure 24.

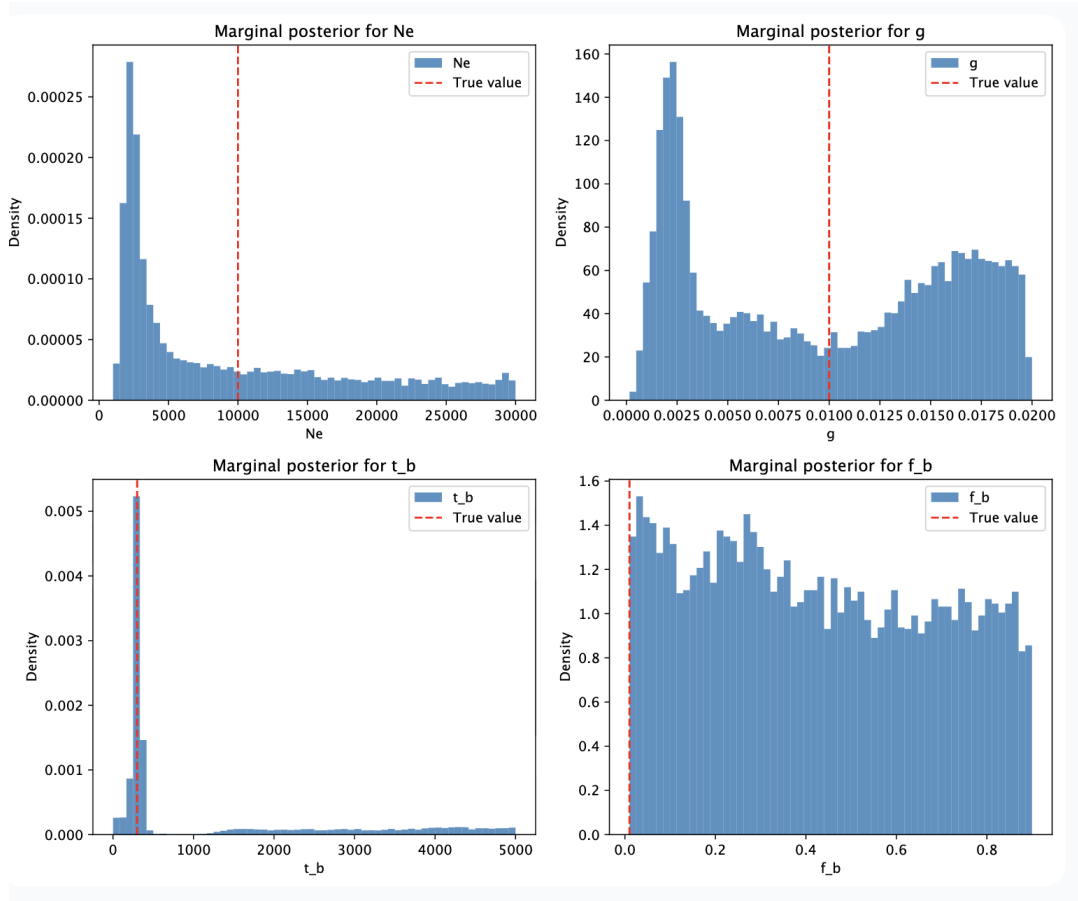


Figure 26: Posterior marginal distributions for (N_e, g, t_b, f_b) using the summary statistics embedding (Parameter setting 2).

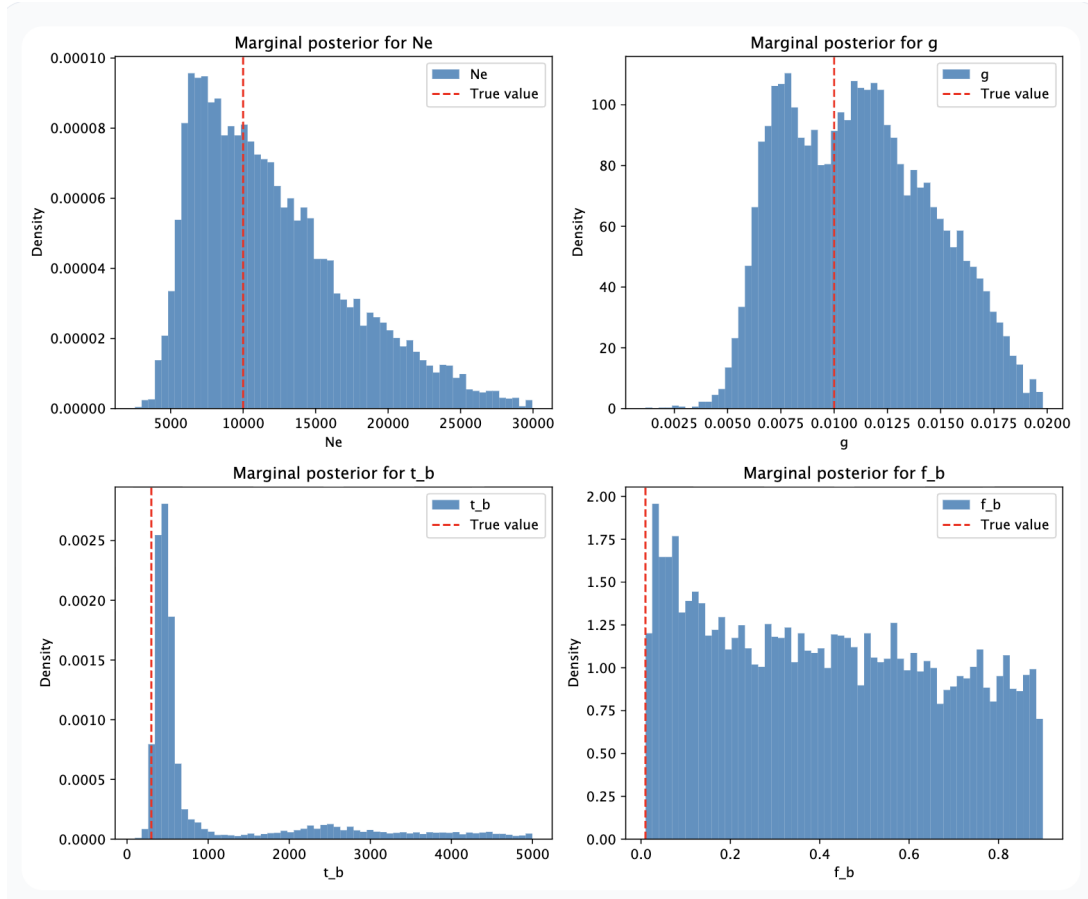


Figure 27: Posterior marginal distributions for (N_e, g, t_b, f_b) using the unsupervised GIN embedding (Parameter setting 2).

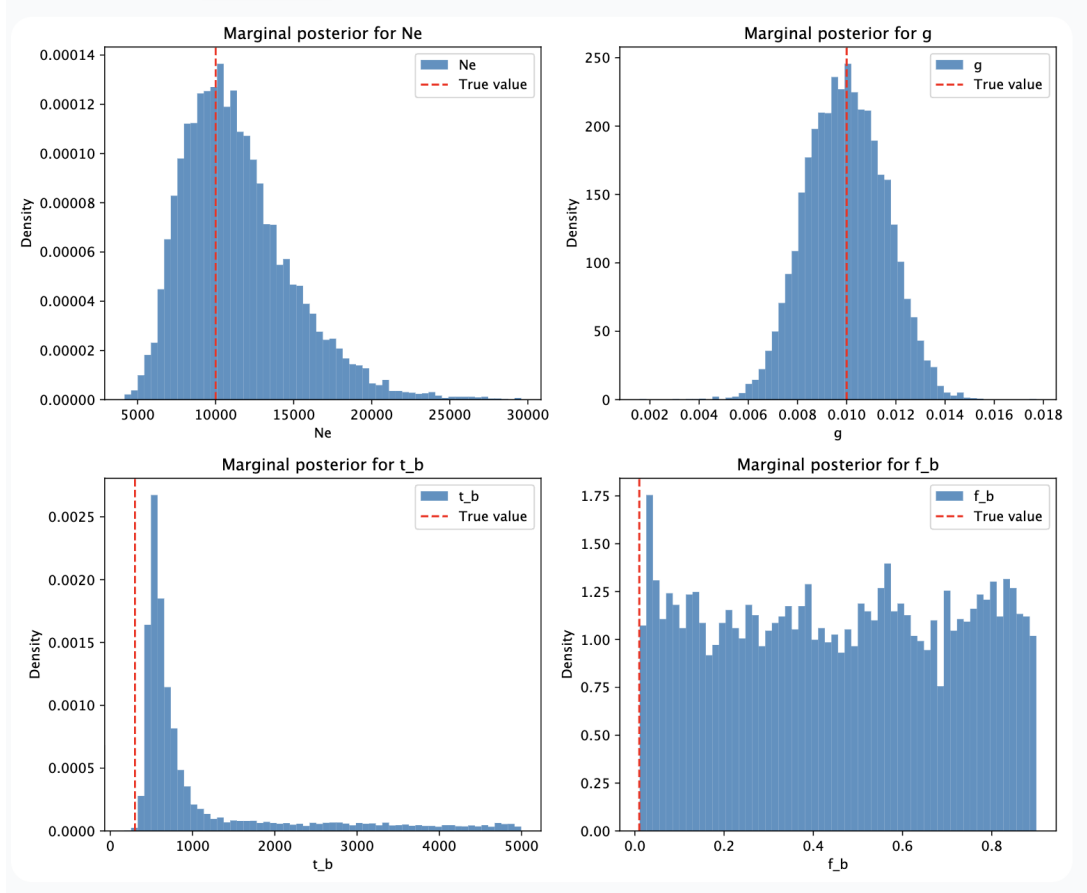


Figure 28: Posterior marginal distributions for (N_e, g, t_b, f_b) using the supervised GIN embedding (Parameter setting 2).

Table 8: True summary statistics for the tree in Parameter setting 2.

	total branch length	tree height	cherries	Sackin	Colless _{norm}
True value	14360.381	1476.792	21	425	0.085

Table 9: Absolute parameter errors for Parameter setting 2. For each model we report $|\Delta N_e|$, $|\Delta g|$, $|\Delta t_b|$, $|\Delta f_b|$, and the total absolute parameter error $\sum_j |\Delta \theta_j|$.

Model	$ \Delta N_e $	$ \Delta g $	$ \Delta t_b $	$ \Delta f_b $	$\sum_j \Delta \theta_j $
Summary stats	8347.520	0.0049	49.963	0.857	8398.345
GNN (unsup.)	1387.626	0.0018	158.416	0.592	1546.635
GNN (sup.)	692.174	0.0029	16.973	0.017	709.167

Table 10: Absolute errors in summary statistics for Parameter setting 2. We report $|s_{\text{pred}} - s_{\text{true}}|$ for each statistic and the total absolute error $\sum_k |s_k|$.

Model	$ \Delta\text{TBL} $	$ \Delta\text{Height} $	$ \Delta\text{cherries} $	$ \Delta\text{Sackin} $	$ \Delta\text{Colless}_{\text{norm}} $	$\sum_k \Delta s_k $
Summary stats	225.470	91.044	2	29	0.030	347.544
GNN (unsup.)	2611.313	1028.416	1	61	0.041	3701.770
GNN (sup.)	569.484	15.456	3	27	0.010	614.950

Table 11: Relative parameter errors (absolute error divided by the true value) for Parameter setting 2, reported as percentages.

Model	$ \Delta N_e /N_e$ [%]	$ \Delta g / g $ [%]	$ \Delta t_b /t_b$ [%]	$ \Delta f_b /f_b$ [%]
Summary stats	83.5	48.8	16.7	8568.6
GNN (unsup.)	13.9	17.7	52.8	5918.7
GNN (sup.)	6.9	29.0	5.7	169.5

Table 12: Relative errors in summary statistics for Parameter setting 2, reported as percentages $|s_{\text{pred}} - s_{\text{true}}|/|s_{\text{true}}|$.

Model	TBL [%]	Height [%]	Cherries [%]	Sackin [%]	Colless _{norm} [%]
Summary stats	1.6	6.2	9.5	6.8	35.2
GNN (unsup.)	18.2	69.6	4.8	14.4	49.0
GNN (sup.)	4.0	1.0	14.3	6.4	11.7

For reference, the mean relative parameter error (averaged over N_e , t_b , and f_b) for this parameter setting is

$$\bar{r}_{\text{param}} = 28.90 \text{ (summary stats), } 19.95 \text{ (GNN unsup.), } 0.61 \text{ (GNN sup.),}$$

and the mean relative summary-statistic error is

$$\bar{r}_{\text{stats}} = 0.12 \text{ (summary stats), } 0.31 \text{ (GNN unsup.), } 0.07 \text{ (GNN sup.).}$$

Results. For this recent, very severe bottleneck, the summary-statistics model reconstructs the tree-level statistics quite accurately but performs poorly on the parameters, especially N_e and f_b . Both GIN-based representations substantially reduce parameter error, with the supervised GIN giving the best overall recovery of (N_e, g, t_b, f_b) . All three methods still struggle to pin down the tiny bottleneck severity $f_b = 0.01$.

6.3 Parameter setting 3: Older moderate bottleneck with growth

True parameters and motivation

As a third configuration we use a larger, growing population with an older, moderately severe bottleneck:

$$\theta_{\text{true}} = (N_e, g, t_b, f_b) = (20,000, 0.015, 800, 0.05).$$

Unlike the recent bottlenecks in Sections 6.1 and 6.2, here all coalescent events in the realised tree occur before the shaded bottleneck interval around $t_b = 800$ (see Figure 30). In other words, the demographic change never affects the genealogy we observe. This case therefore illustrates how the models behave when the bottleneck parameters (t_b, f_b) are effectively unidentifiable from the tree.

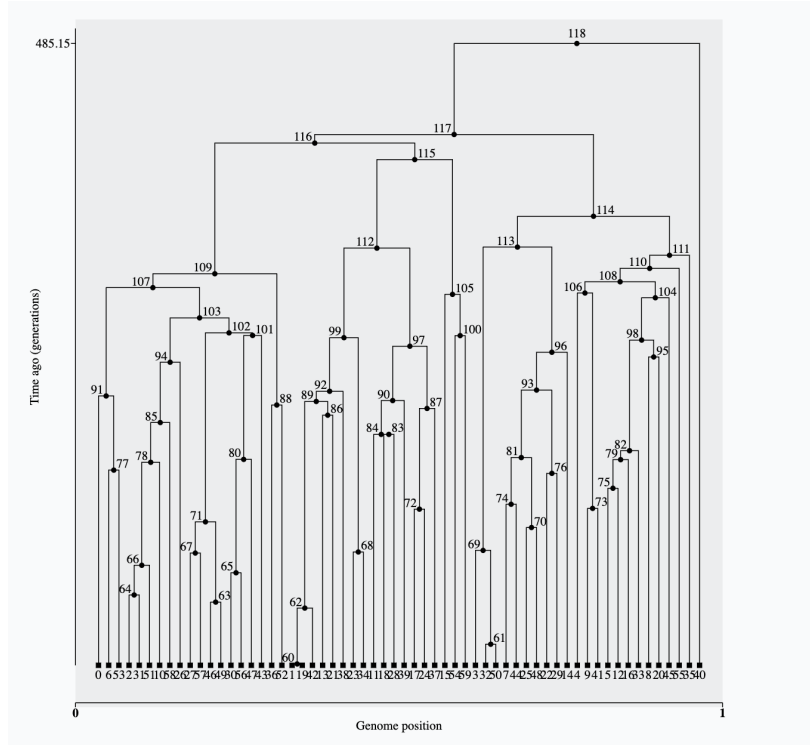


Figure 29: Phylogeny simulated at $\theta_{\text{true}} = (20,000, 0.015, 800, 0.05)$.

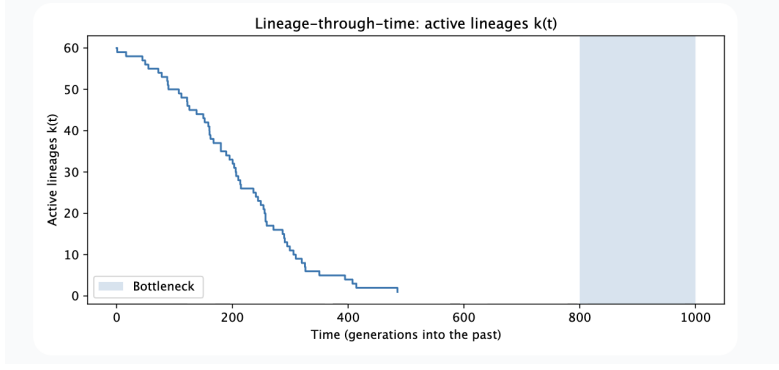


Figure 30: Lineage-through-time plot for the tree in Figure 29.

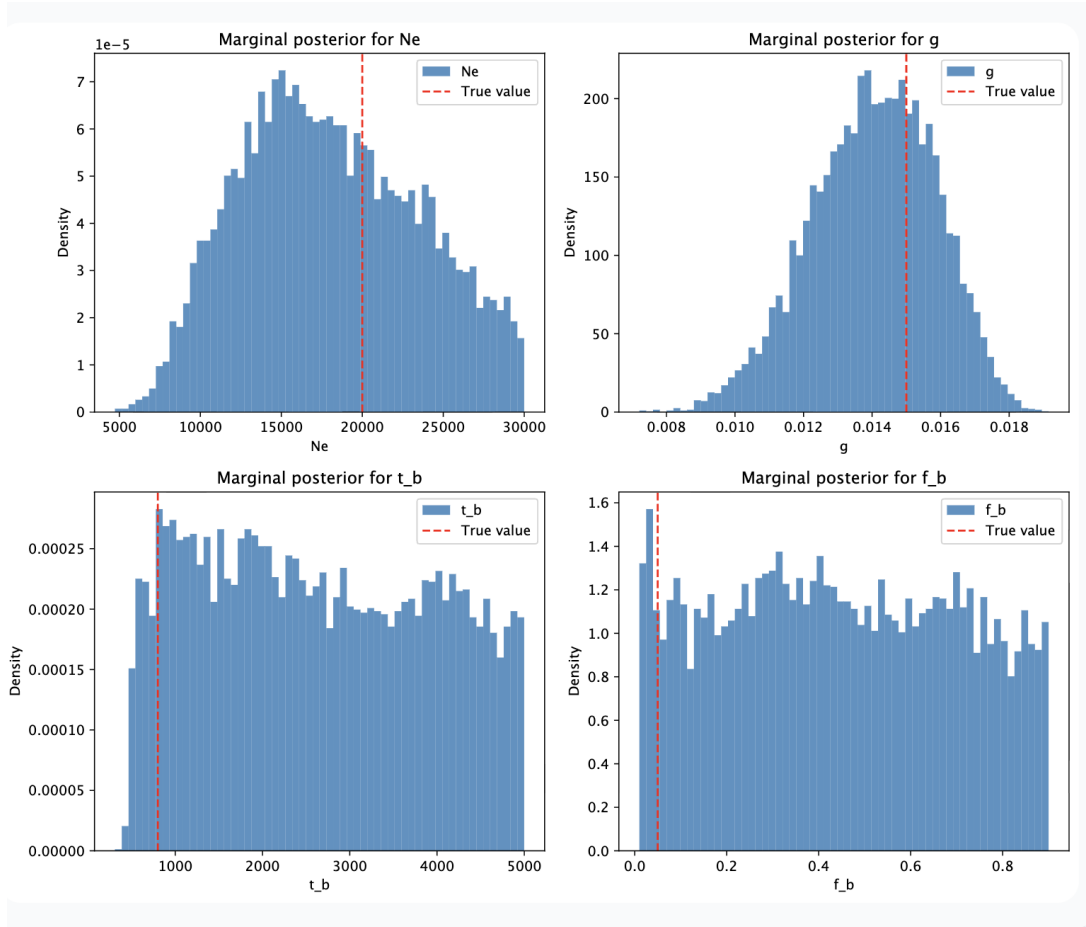


Figure 31: Posterior marginal distributions for (N_e, g, t_b, f_b) using the summary-statistics embedding.

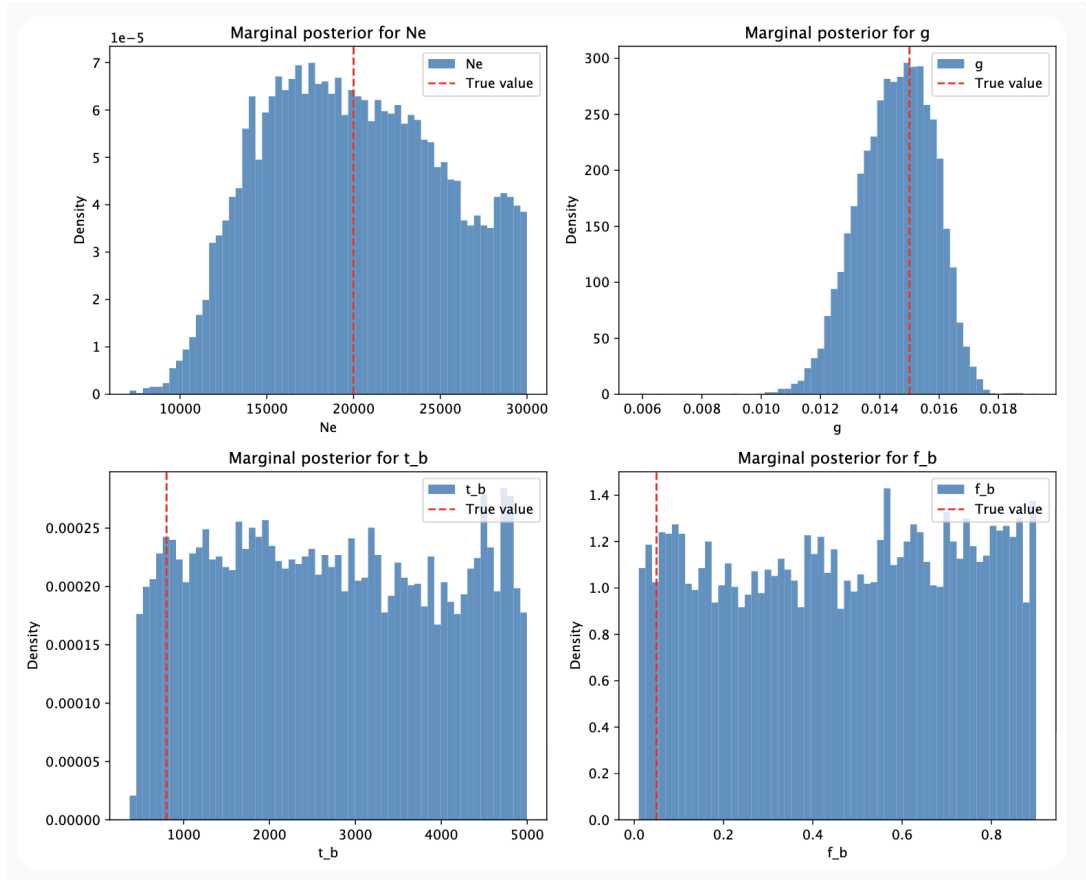


Figure 32: Posterior marginals using the unsupervised GIN embedding.

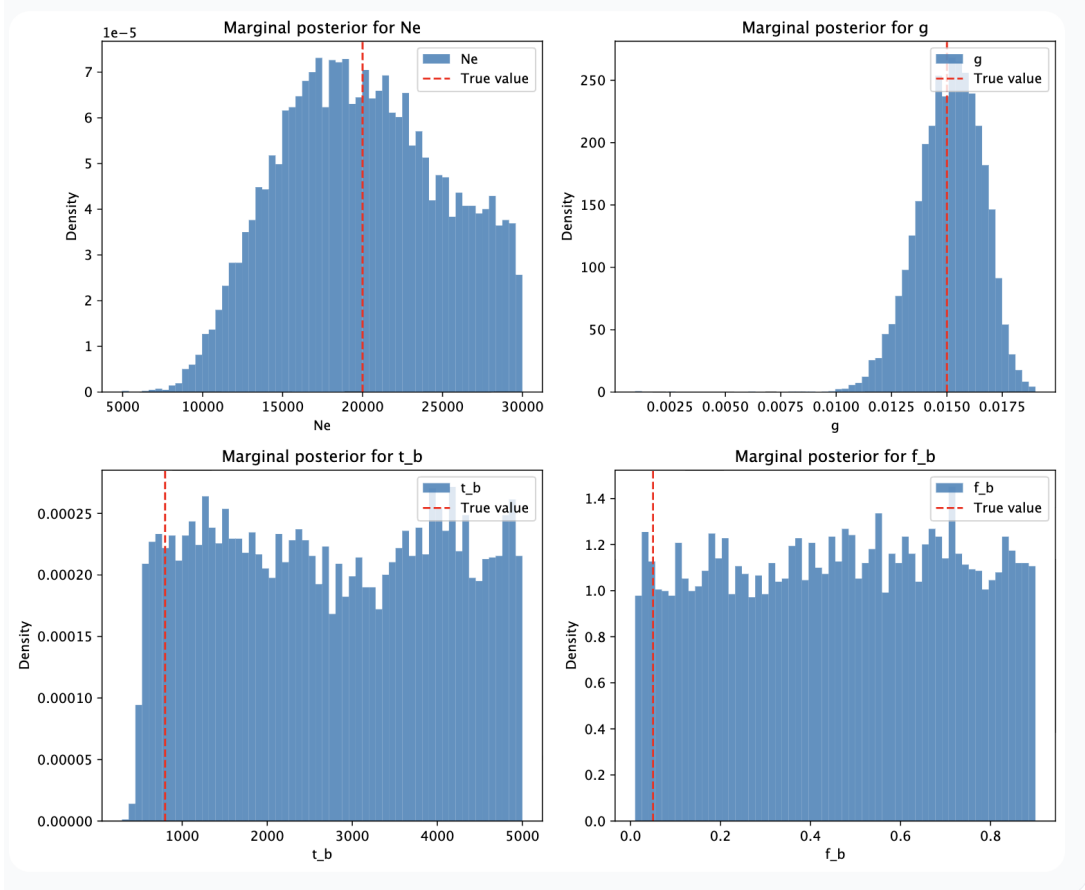


Figure 33: Posterior marginals using the supervised GIN embedding.

Table 13: Summary statistics for the tree in Figure 29.

	total branch length	tree height	cherries	Sackin	Colless _{norm}
True value	12171.655	452.997	21	422	0.078

Table 14: Absolute parameter errors for Parameter setting 3.

Model	$ \Delta N_e $	$ \Delta g $	$ \Delta t_b $	$ \Delta f_b $	$\sum_j \Delta \theta_j $
Summary stats	2229.207	0.0001	1429.820	0.024	3659.051
GNN (unsup.)	5961.531	0.0013	47.275	0.0119	6008.820
GNN (sup.)	5597.245	0.0011	4069.846	0.595	9667.688

Table 15: Absolute errors in summary statistics for Parameter setting 3.

Model	$ \Delta \text{TBL} $	$ \Delta \text{Height} $	$ \Delta \text{cherries} $	$ \Delta \text{Sackin} $	$ \Delta \text{Colless}_{\text{norm}} $	$\sum_k \Delta s_k $
Summary stats	96.495	41.668	3	26	0.007	167.170
GNN (unsup.)	1504.896	43.188	0	63	0.040	1611.124
GNN (sup.)	582.246	18.732	1	14	0.021	615.999

Table 16: Relative parameter errors (percent) for Parameter setting 3.

Model	$ \Delta N_e /N_e$ [%]	$ \Delta g / g $ [%]	$ \Delta t_b /t_b$ [%]	$ \Delta f_b /f_b$ [%]
Summary stats	11.1	0.8	178.7	47.7
GNN (unsup.)	29.8	8.8	5.9	23.7
GNN (sup.)	28.0	7.0	508.7	1190.3

Table 17: Relative errors (percent) in summary statistics for Parameter setting 3.

Model	TBL [%]	Height [%]	Cherries [%]	Sackin [%]	Colless _{norm} [%]
Summary stats	0.8	9.2	14.3	6.2	9.0
GNN (unsup.)	12.4	9.5	0.0	14.9	51.5
GNN (sup.)	4.8	4.1	4.8	3.3	26.9

For this parameter setting, the mean relative parameter error (averaged over N_e , t_b , and f_b) is

$$\bar{r}_{\text{param}} = 0.79 \text{ (summary stats), } 0.20 \text{ (GNN unsup.), } 5.76 \text{ (GNN sup.),}$$

and the mean relative summary-statistic error is

$$\bar{r}_{\text{stats}} = 0.08 \text{ (summary stats), } 0.18 \text{ (GNN unsup.), } 0.09 \text{ (GNN sup.).}$$

Results. Because the realised tree has already coalesced before the bottleneck window, the data carry almost no information about t_b and f_b . Accordingly, all three methods produce broad, almost uniform posteriors over these two parameters, and the resulting point estimates can lie far from the truth, leading to very large relative errors, especially for the summary-statistics model and the supervised GIN. In contrast, N_e and g remain better constrained, with the unsupervised GIN achieving the lowest overall parameter error in this setting.

6.4 Parameter setting 4: Large population with very recent severe bottleneck

True parameters and motivation

Finally, we consider the largest, fastest-growing population with an extremely recent, very severe bottleneck:

$$\theta_{\text{true}} = (N_e, g, t_b, f_b) = (25,000, 0.02, 200, 0.01).$$

Most of the demographic signal is concentrated close to the present, making this a challenging case where the models must disentangle rapid recent changes from the rest of the genealogy.

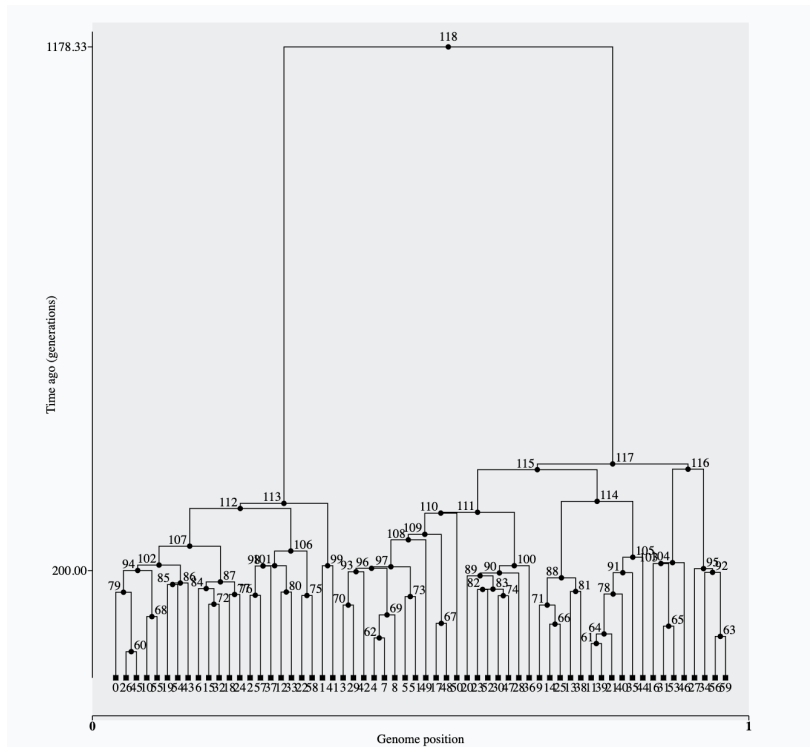


Figure 34: Phylogeny simulated at $\theta_{\text{true}} = (25,000, 0.02, 200, 0.01)$.

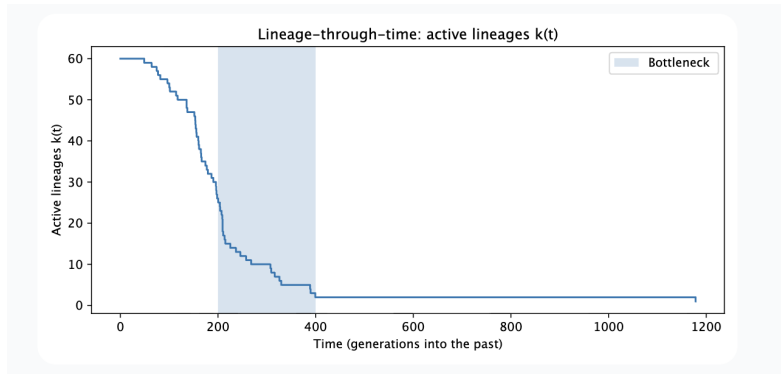


Figure 35: Lineage-through-time plot for the tree in Figure 34.

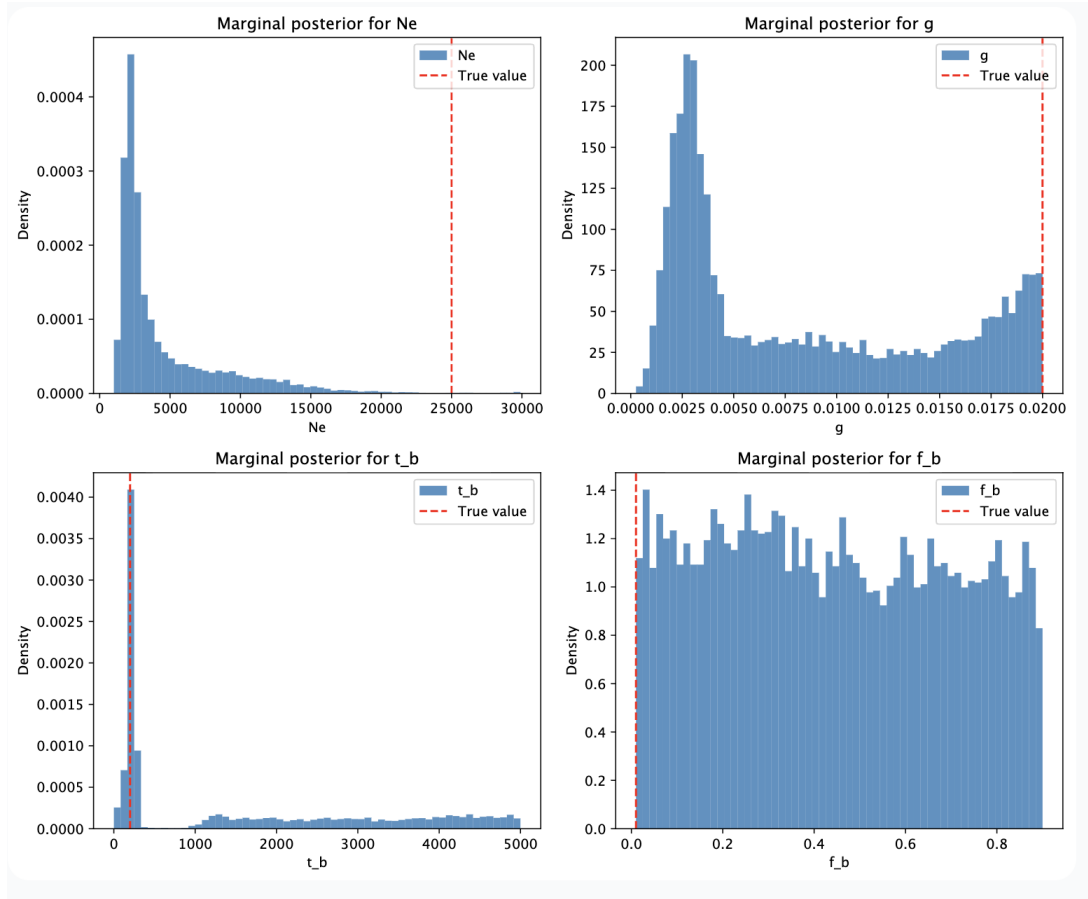


Figure 36: Posterior marginal distributions for (N_e, g, t_b, f_b) using the summary-statistics embedding.

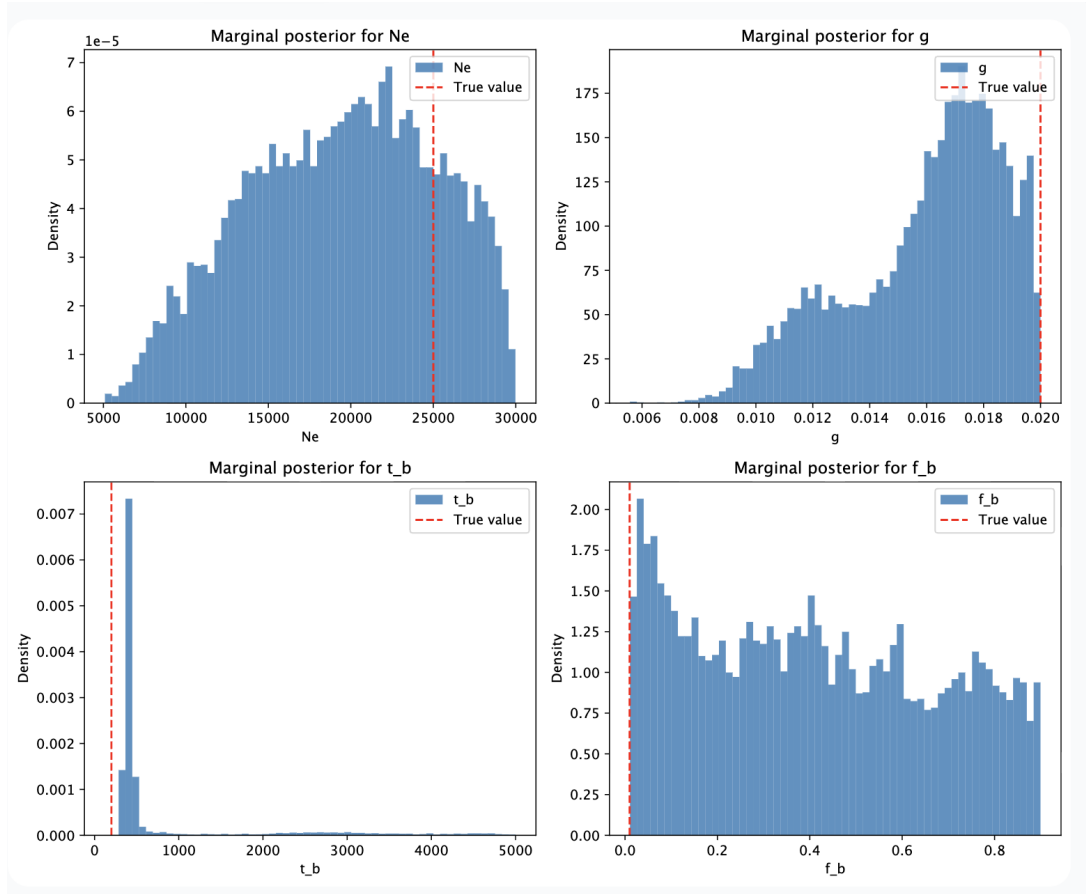


Figure 37: Posterior marginals using the unsupervised GIN embedding.

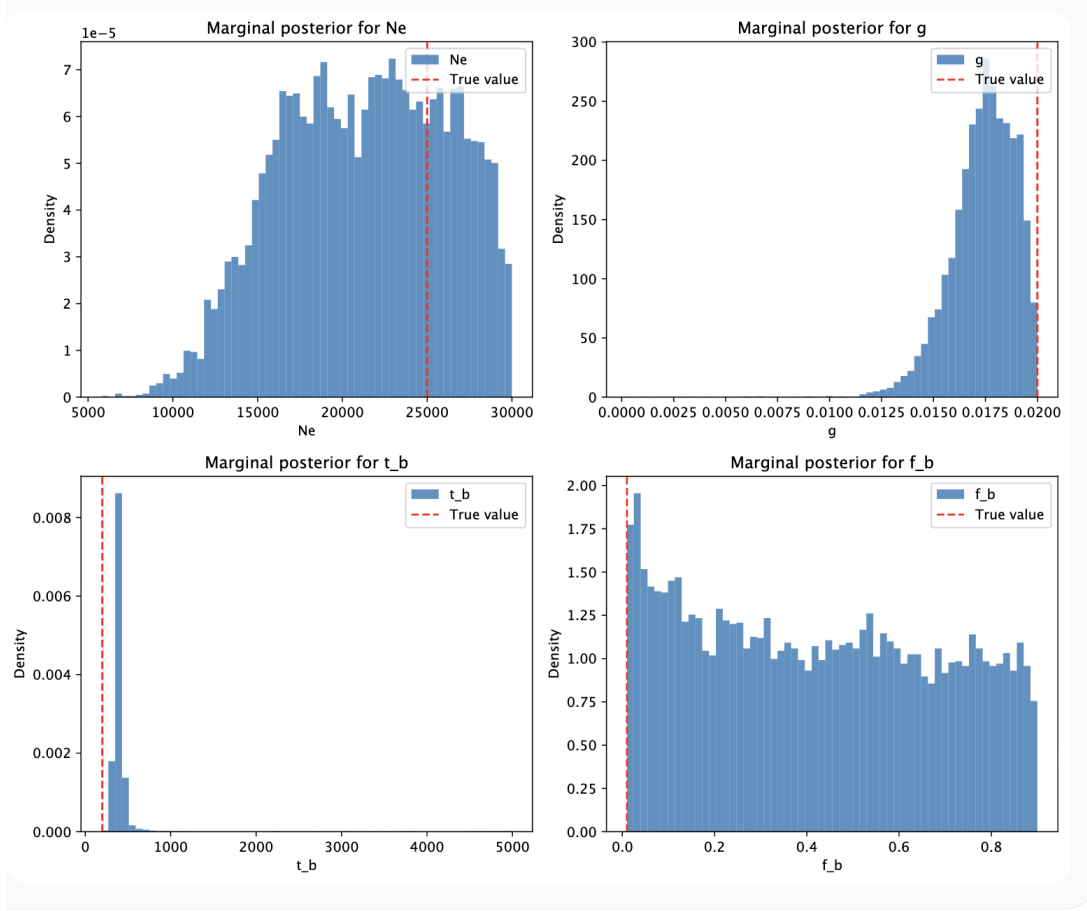


Figure 38: Posterior marginals using the supervised GIN embedding.

Table 18: Summary statistics for the tree in Figure 34.

	total branch length	tree height	cherries	Sackin	Colless _{norm}
True value	14183.294	1091.853	19	482	0.101

Table 19: Absolute parameter errors for Parameter setting 4.

Model	$ \Delta N_e $	$ \Delta g $	$ \Delta t_b $	$ \Delta f_b $	$\sum_j \Delta \theta_j $
Summary stats	23181.350	0.0109	29.481	0.860	23211.701
GNN (unsup.)	3691.871	0.0002	114.349	0.0172	3806.238
GNN (sup.)	2321.043	0.0003	112.939	0.0087	2433.991

Table 20: Absolute errors in summary statistics for Parameter setting 4.

Model	$ \Delta \text{TBL} $	$ \Delta \text{Height} $	$ \Delta \text{cherries} $	$ \Delta \text{Sackin} $	$ \Delta \text{Colless}_{\text{norm}} $	$\sum_k \Delta s_k $
Summary stats	973.006	95.616	1	32	0.005	1101.627
GNN (unsup.)	509.061	273.041	1	24	0.016	807.117
GNN (sup.)	86.145	324.101	1	57	0.025	468.271

Table 21: Relative parameter errors (percent) for Parameter setting 4.

Model	$ \Delta N_e /N_e$ [%]	$ \Delta g / g $ [%]	$ \Delta t_b /t_b$ [%]	$ \Delta f_b /f_b$ [%]
Summary stats	92.7	54.3	14.7	8598.0
GNN (unsup.)	14.8	1.0	57.2	172.3
GNN (sup.)	9.3	1.4	56.5	86.7

Table 22: Relative errors (percent) in summary statistics for Parameter setting 4.

Model	TBL [%]	Height [%]	Cherries [%]	Sackin [%]	Colless _{norm} [%]
Summary stats	6.9	8.8	5.3	6.6	4.7
GNN (unsup.)	3.6	25.0	5.3	5.0	16.3
GNN (sup.)	0.6	29.7	5.3	11.8	25.0

For this parameter setting, the mean relative parameter error (averaged over N_e , t_b , and f_b) is

$$\bar{r}_{\text{param}} = 29.02 \text{ (summary stats), } 0.81 \text{ (GNN unsup.), } 0.51 \text{ (GNN sup.),}$$

and the mean relative summary-statistic error is

$$\bar{r}_{\text{stats}} = 0.06 \text{ (summary stats), } 0.11 \text{ (GNN unsup.), } 0.14 \text{ (GNN sup.).}$$

Results. In this largest-population, very recent bottleneck scenario, the summary-statistics embedding matches the tree statistics reasonably well but fails badly on N_e and f_b , placing most posterior mass far from the true values. Both GIN-based models correct much of this bias and achieve substantially smaller parameter errors overall, with the supervised GIN giving the best aggregate recovery of (N_e, g, t_b, f_b) . However, the bottleneck severity remains difficult to estimate accurately for all three representations.

7 Conclusion

Across all settings, two themes emerge. First, representation choice has a strong and systematic impact on which aspects of demography are recoverable. Effective population size N_e and growth rate g are generally better constrained than the bottleneck time and severity (t_b, f_b) , and the three encoders trade off bias in these parameters in different ways. The summary-statistics representation tends to match tree-level statistics well and often produces accurate estimates of N_e (and g when it is non-zero), but it struggles with t_b and f_b , sometimes placing most posterior mass far from the true bottleneck (e.g. Sections 6.1 and 6.2). The unsupervised GIN generally yields the smallest mean relative

parameter error when the bottleneck actually affects the genealogy (Parameters 2 and 3), and consistently produces posteriors whose implied trees are close to the observed one in terms of summary statistics. The supervised GIN can perform very well in some regimes (especially the recent, severe bottlenecks in Sections 6.2 and 6.4), but degrades badly when bottleneck parameters are weakly identified, indicating a higher sensitivity to model misspecification and training choices.

Second, the experiments expose clear identifiability limits of the bottleneck parameters. In Parameter setting 3 the realised tree has already coalesced before the bottleneck window, so the data contain essentially no information about t_b and f_b . All three methods then correctly respond by producing broad, almost uniform posteriors over these parameters, with large relative errors driven not by a failure of the inference model but by an inherent lack of signal. Even in settings where the bottleneck is realised (Parameters 1, 2, and 4), the tiny severities ($f_b = 0.01$) remain difficult to recover: posteriors for f_b are typically wide and often biased, reflecting the limited information that a single tree carries about short, sharp historical events.

Taken together, these results show that NPE is a viable and flexible approach for coalescent-based phylogenetic inference, but that its practical performance is tightly coupled to the chosen tree representation and to the intrinsic identifiability of the demographic parameters. Hand-crafted summary statistics remain a surprisingly strong baseline, especially for N_e and for matching simple tree-shape summaries. Unsupervised GNN embeddings offer a robust alternative that often improves parameter recovery without requiring labels during representation learning. Supervised GNNs can yield the best performance in favourable regimes but behave less predictably when the data are uninformative about some parameters.

8 Future work

There are several directions in which this work could be extended. First, it would be natural to benchmark the flow-based NPE posteriors against established ABC variants such as ABC-SMC and ABC-MCMC under matched simulation settings, to quantify the trade-offs in accuracy and computational cost. Second, the current study is restricted to simulated trees; an important next step is to apply the full pipeline to empirical phylogenies (for example, SARS-CoV or influenza datasets) and assess whether the inferred bottleneck histories are biologically plausible. Finally, the prototype *PhyloInference Play* dashboard could be developed into a public-facing web tool with additional visualisations, support for user-uploaded trees, and configurable demographic models, making simulation-based phylogenetic inference more accessible to practitioners.

References

- Beaumont, Mark A., Wenyang Zhang, and David J. Balding (2002). “Approximate Bayesian computation in population genetics”. In: *Genetics* 162, pp. 2025–2035.
- Cmero, Matthew (2015). *Building Phylogenetic Trees with Binary Traits*. Blog post; accessed 2025-08-16. URL: <https://mcmero.github.io/blog/Perfect-phylogeny/>.
- Cranmer, Kyle, Johann Brehmer, and Gilles Louppe (2020). “The frontier of simulation-based inference”. In: *Proceedings of the National Academy of Sciences* 117.48, pp. 30055–30062.
- Crow, James F. (2008). “Wright and Fisher on Inbreeding and Random Drift”. In: *Genetics*.
- Dax, Michael et al. (2023). “Flow Matching for Scalable Simulation-Based Inference”. In: *Advances in Neural Information Processing Systems (NeurIPS)*.
- Deistler, Michael, Pedro J. Gonçalves, and Jakob H. Macke (2022). “Truncated Proposals for Scalable and Hassle-Free Simulation-Based Inference”. In: *Advances in Neural Information Processing Systems (NeurIPS)*.
- Duvenaud, David K. et al. (2015). “Convolutional Networks on Graphs for Learning Molecular Fingerprints”. In: Curran Associates, Inc.
- Fisher, Ronald A. (1930). *The Genetical Theory of Natural Selection*. Oxford: Oxford University Press.
- Flagel, Lex, Yaniv Brandvain, and Daniel R. Schrider (2019). “The Unreasonable Effectiveness of Convolutional Neural Networks in Population Genetic Inference”. In: *Molecular Biology and Evolution* 36.2, pp. 220–238.
- Geffner, Tom, George Papamakarios, and Andriy Mnih (2023). “Compositional Score Modeling for Simulation-Based Inference”. In: *Proceedings of the 40th International Conference on Machine Learning (ICML)*.
- Greenberg, David, Marcel Nonnenmacher, and Jakob H. Macke (2019). “Automatic Posterior Transformation for Likelihood-Free Inference”. In: *Proceedings of the 36th International Conference on Machine Learning (ICML)*.
- Lueckmann, Jan-Matthis et al. (2017). “Flexible Statistical Inference for Mechanistic Models of Neural Dynamics”. In: *Advances in Neural Information Processing Systems (NeurIPS)*.
- Marjoram, Paul et al. (2003). “Markov chain Monte Carlo without likelihoods”. In: *Proceedings of the National Academy of Sciences* 100.26, pp. 15324–15328.
- msprime developers (2025). *msprime on PyPI*. Accessed 2025-08-16. URL: <https://pypi.org/project/msprime/>.
- Papamakarios, George and Iain Murray (2016). “Fast ϵ -free Inference of Simulation Models with Bayesian Conditional Density Estimation”. In: *Advances in Neural Information Processing Systems (NeurIPS)*.

- Paszke, Adam et al. (2019). “PyTorch: An Imperative Style, High-Performance Deep Learning Library”. In: *Advances in Neural Information Processing Systems (NeurIPS)*.
- Peng, Nanyun et al. (2017). “Cross-Sentence N-ary Relation Extraction with Graph LSTMs”. In: *Transactions of the Association for Computational Linguistics* 5, pp. 101–115. DOI: 10.1162/tacl_a_00049.
- Pritchard, Jonathan K. et al. (1999). “Population Growth of Human Y Chromosomes: A Study of Y Chromosome Microsatellites”. In: *Molecular Biology and Evolution* 16.12, pp. 1791–1798.
- PyTorch Team (2025). *PyTorch Documentation (Stable)*. Accessed 2025-08-16. URL: <https://pytorch.org/docs/stable/>.
- Queau, Pierre-Yves, Wojbor Woyczyński, and Alan Lerner (2016). “Coalescing Stochastic Processes in Retrieval from Semantic Memory”. In: *Mathematica Applicanda* 43.2.
- Radev, Stefan T. et al. (2020). “BayesFlow: Learning Complex Stochastic Models with Invertible Neural Networks”. In: *IEEE Transactions on Neural Networks and Learning Systems*.
- SBI Developers (2025). *sbi: Simulation-Based Inference Toolkit—Documentation*. Accessed 2025-08-16. URL: <https://sbi.readthedocs.io/en/latest/>.
- Sheehan, Sara and Yun S. Song (2016). “Deep Learning for Population Genetic Inference”. In: *PLOS Computational Biology* 12.3, e1004845.
- Sisson, Scott A., Yanan Fan, and Mark M. Tanaka (2007). “Sequential Monte Carlo without Likelihoods”. In: *Proceedings of the National Academy of Sciences* 104.6, pp. 1760–1765.
- Tejero-Cantero, Álvaro et al. (2020). “sbi: A Toolkit for Simulation-Based Inference”. In: *Journal of Open Source Software* 6.67, p. 2505.
- Tree Sequence Developers (2024). *msprime Documentation*. Accessed 2025-08-16. URL: <https://tskit.dev/msprime/>.
- Wang, Xiaolong, Yufei Ye, and Abhinav Gupta (2018). “Zero-Shot Recognition via Semantic Embeddings and Knowledge Graphs”. In.
- Wang, Zichao, Jun Wang, and Marios Kourakos (2021). “Automatic Inference of Demographic Parameters Using Generative Adversarial Networks”. In: *Molecular Ecology Resources*.
- Weng, Lilian (2018). *Flow-based Deep Generative Models*. URL: <https://lilianweng.github.io/posts/2018-10-13-flow-models/>.
- Wilson, Huon (2020). *Do I know you? Flexible unsupervised and semi-supervised graph models with Deep Graph Infomax*. URL: <https://medium.com/stellargraph/do-i-know-you-flexible-unsupervised-and-semi-supervised-graph-models-with-deep-graph-infomax-96fbfd63ec31>.
- Wright, Sewall (1931). “Evolution in Mendelian Populations”. In: *Genetics* 16.2, pp. 97–159.

- Yin, Peiqi et al. (2023). “DGI: Easy and Efficient Inference for GNNs”. In: *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*. ACM.
- Zhang, Xiao-Meng et al. (2021). “Graph Neural Networks and Their Current Applications in Bioinformatics”. In: *Frontiers in Genetics*. ISSN: 1664-8021.
- Zitnik, Marinka, Monica Agrawal, and Jure Leskovec (2018). “Modeling Polypharmacy Side Effects with Graph Convolutional Networks”. In: *Bioinformatics* 34.

Appendix A: Code Listings

CODE SNIPPETS FOR IMPORTANT FUNCTIONS

```
def simulate_ts_2(Ne: float, g: float, cfg: SimConfig) ->
tskit.TreeSequence:
    dem = msprime.Demography()
    dem.add_population(initial_size=Ne, growth_rate=g)

    ts = msprime.sim_ancestry(
        samples=cfg.sample_size,
        sequence_length=cfg.sequence_length,
        recombination_rate=cfg.recomb_rate,
        demography=dem,
        random_seed=cfg.random_seed,
    )
    return ts

def simulate_ts_4_bottleneck(Ne, g, tb, fb, cfg, dur=200.0):
    dem = msprime.Demography()
    dem.add_population(initial_size=Ne, growth_rate=g)
    dem.add_population_parameters_change(time=tb,
initial_size=Ne*fb, growth_rate=0.0)
    dem.add_population_parameters_change(time=tb+dur,
initial_size=Ne*math.exp(g*(tb+dur)), growth_rate=g)
    return msprime.sim_ancestry(
        samples=cfg.sample_size,
        sequence_length=cfg.sequence_length,
        recombination_rate=cfg.recomb_rate,
        demography=dem,
        random_seed=cfg.random_seed,
    )

def compute_stats(ts: tskit.TreeSequence) -> dict:
    t = ts.first() # single tree (recomb_rate=0)
    n = ts.num_samples

    # Total branch length & height
    tbl = float(t.total_branch_length)
    root = t.root
    tree_height = float(ts.node(root).time) if root != tskit.NULL else
float("nan")

    # Prepare helpers
    samples = set(ts.samples())

    # Cherries: internal nodes whose two children are both leaves
(samples)
    cherries = 0
    for u in t.nodes():
        if u == root or t.is_leaf(u):
            continue
```

```

        if _is_binary(t, u):
            a, b = _children(t, u)
            if t.is_leaf(a) and t.is_leaf(b) and (a in samples) and (b
in samples):
                cherries += 1

    # Sackin (raw): sum of depths over leaves
    sackin = 0
    for leaf in samples:
        sackin += _leaf_depth(t, leaf)

    # Colless normalized: sum_{internal} |n_L - n_R| / max
    colless = 0
    for u in t.nodes():
        if u == root or t.is_leaf(u):
            continue
        if _is_binary(t, u):
            a, b = _children(t, u)
            nL = _num_descendant_samples(t, a)
            nR = _num_descendant_samples(t, b)
            colless += abs(nL - nR)

    # Normalization
    colless_max = (n - 1) * (n - 2) / 2 if n >= 3 else 1.0
    colless_norm = float(colless / colless_max)

    return {
        "total_branch_length": float(tbl),
        "tree_height": float(tree_height),
        "cherries": int(cherries),
        "sackin": int(sackin),
        "colless_norm": float(colless_norm),
    }

def mlp(d_in, d_hid, d_out):
    return nn.Sequential(nn.Linear(d_in, d_hid), nn.ReLU(),
nn.Linear(d_hid, d_out))

class GINEncoder(nn.Module):
    def __init__(self, in_dim=5, hid=64, out_dim=64, train_eps=True):
        super().__init__()
        self.conv1 = GINConv(mlp(in_dim, hid, hid),
train_eps=train_eps)
        self.bn1 = BatchNorm(hid)
        self.conv2 = GINConv(mlp(hid, hid, hid),
train_eps=train_eps)
        self.bn2 = BatchNorm(hid)
        self.lin_g = nn.Linear(hid, out_dim)

    def forward(self, x, edge_index, batch=None):

```

```

        h = F.relu(self.bn1(self.conv1(x, edge_index)))
        h = F.relu(self.bn2(self.conv2(h, edge_index)))
        return h

    def graph_embed(self, h, batch):
        g = global_add_pool(h, batch)
        return self.lin_g(g)

class DGIModel(nn.Module):
    def __init__(self, in_dim=5, hid=64, out_dim=64):
        super().__init__()
        self.enc = GINEncoder(in_dim, hid, out_dim)

        self.dgi = DeepGraphInfomax(
            hidden_channels=hid,
            encoder=lambda x, edge_index, batch=None: self.enc(x,
edge_index, batch),
            summary=lambda z, *args, **kwargs:
torch.sigmoid(z.mean(dim=0, keepdim=True)),
            corruption=lambda x, edge_index, batch=None:
(x[torch.randperm(x.size(0))], edge_index),
            )

    def loss(self, data):
        pos_z, neg_z, summary = self.dgi(data.x, data.edge_index,
batch=data.batch)
        return self.dgi.loss(pos_z, neg_z, summary)

    @torch.no_grad()
    def encode_graphs(self, data):
        h = self.enc(data.x, data.edge_index, data.batch)    # node
embeddings
        z = self.enc.graph_embed(h, data.batch)              # graph
embeddings
        return z

def train_dgi(model: DGIModel, loader: DataLoader, epochs=30, lr=1e-3,
wd=1e-5):
    model = model.to(DEVICE)
    opt = torch.optim.Adam(model.parameters(), lr=lr, weight_decay=wd)
    for ep in range(1, epochs+1):
        model.train()
        tot=0.0
        for batch in loader:
            batch = batch.to(DEVICE)
            loss = model.loss(batch)
            opt.zero_grad(); loss.backward(); opt.step()
            tot += loss.item() * batch.num_graphs
        if ep % 5 == 0 or ep == 1:

```

```

        print(f"[DGI ep {ep:03d}] loss
{tot/len(loader.dataset):.4f}")
    return model

prior_bounds = {
    "Ne": (1_000.0, 30_000.0),
    "g": (0.0, 0.02),
    "t_b": (0.0, 5_000.0),
    "f_b": (0.01, 0.9),
}

low = torch.tensor(
    [prior_bounds["Ne"][0],
    prior_bounds["g"][0],
    prior_bounds["t_b"][0],
    prior_bounds["f_b"][0]],
    dtype=torch.float32,
)
high = torch.tensor(
    [prior_bounds["Ne"][1],
    prior_bounds["g"][1],
    prior_bounds["t_b"][1],
    prior_bounds["f_b"][1]],
    dtype=torch.float32,
)

prior = BoxUniform(low=low, high=high)

inference = NPE(prior=prior, density_estimator="nsf")
density_estimator = inference.append_simulations(theta_train,
x_train).train(
    max_num_epochs=300,
    show_train_summary=False,
)
posterior = inference.build_posterior(density_estimator)

```